UNIT –3

➢ SYNTAX –DIRECTED DEFINITION

- SDD (Syntax-Directed Definition) is a method used in compiler design to specify the semantics (meaning) of a programming language by attaching attributes and semantic rules to the grammar productions.
- Attributes are associated with grammar symbols (terminals and non-terminals) and rules are associated with productions that compute the values of these attributes.
- An attribute has a name and an associated value: a string, a number, a type, a memory location, an assigned register, strings.
- If X is a symbol and 'a' is one of its attributes, then we write X.a to denote the value of a at a particular parse-tree node labeled X.
- The attributes are evaluated by the semantic rules attached to the productions.

Example:

E → E1 + T { E.val = E1.val + T.val }
E → T { E.val = T.val }
T → num { T.val = num.lexval }

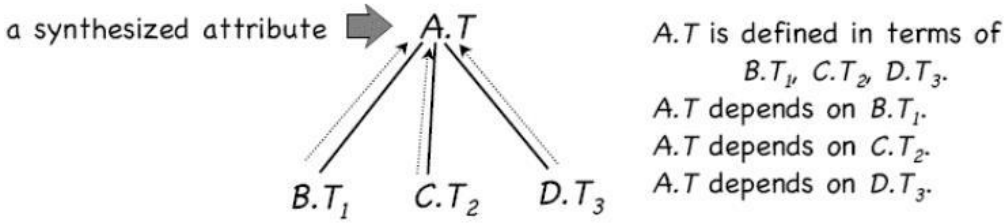
SDDs are highly readable and give high-level specifications for translations. But they hide many implementation details. For example, they do not specify order of evaluation of semantic actions.

➢ Attributes – Types

In compiler design, Syntax Directed Translation calculates the value of the attribute of the node present in the parsing tree. The Syntax Directed Definition (SDD) uses two types of attributes, which are used to determine the value of the attribute at the node.

1. Synthesized Attribute

The value of a synthesized attribute is calculated from the attributes of its children.
Information flows from child → parent.
It is commonly used in bottom-up parsing.
Consider a grammar for arithmetic expressions: E → E1 + T { E.val = E1.val + T.val }. Here, E.val (the value of the expression) is a synthesized attribute because its value is computed from the values of its child nodes, E.val and T.val.

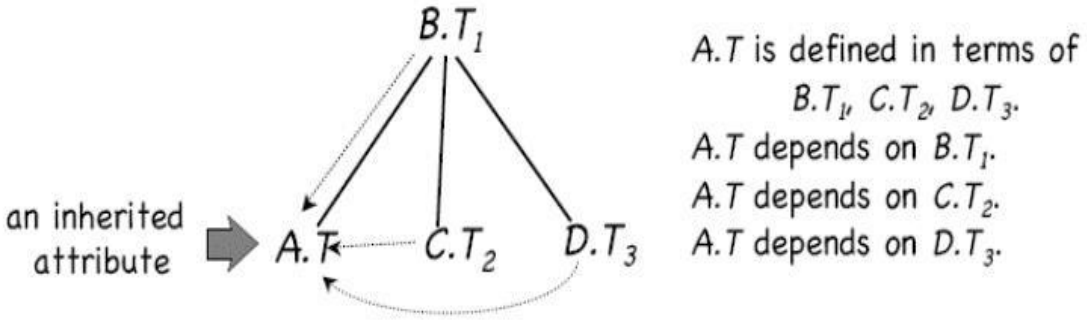


2. Inherited Attribute

The value of an inherited attribute is calculated from the attributes of the parent or siblings.
Information flows from parent/sibling → child.
It is useful for passing contextual information such as type, scope, or symbol-table information.

This type of attribute is mainly associated with top-down parsing

Consider a grammar where a type is propagated down the tree: D → T id { id.type = T.type }. Here, id.type (the type of the identifier) is an inherited attribute because its value is determined by T.type, which comes from its sibling node T.



Feature	Synthesized Attribute	Inherited Attribute
Definition	Value is computed from the children of a node	Value is computed from the parent or siblings
Information flow	Child → Parent	Parent/Sibling → Child
Direction	Usually bottom-up ↑	Usually top-down ↓
Used in	S-attributed SDD	L-attributed SDD
Example	`E.val = E1.val + T.val`	`L.in = T.type`
Common parsing association	Bottom-up parsing such as LR	Top-down parsing such as LL
Dependency	Depends on attributes of child nodes	Depends on attributes of parent/sibling nodes

➢ Classes or Types of SDD

1. S-Attributed Definitions

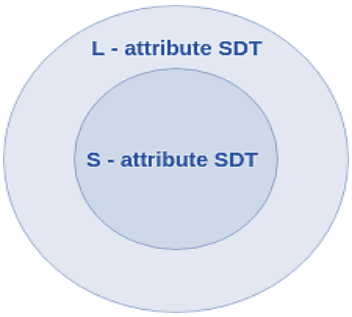
If every attribute of an SDD is *synthesized*, it is S-attributed. It means its value is determined solely by the attribute values of its children in the parse tree.

S-attributed SDDs are well-suited for bottom-up parsing techniques like LR parsing.

Example:

```
E → E1 + T    { E.val = E1.val + T.val }
E → T         { E.val = T.val }
T → num       { T.val = num.lexval }
```

Here, E.val and T.val are synthesized attributes.



2. L-Attributed Definitions

L-attributed definitions are the second type of SDD.

An L-attributed definition uses both *synthesized and restricted inherited attributes*. The key restriction is that for any production, an inherited attribute of a symbol can only depend on its parent and its left siblings in the parse tree.

L-attributed definitions are more flexible and are often used with top-down parsing methods

Attributes are evaluated using a *depth-first, left-to-right* traversal of the parse tree.

Example:

```
D → T L          { L.in = T.type }
T → int          { T.type = int }
T → float        { T.type = float }
L → id           { id.type = L.in }
L → id , L1      { id.type = L.in
                   L1.in = L.in }
```

Here:

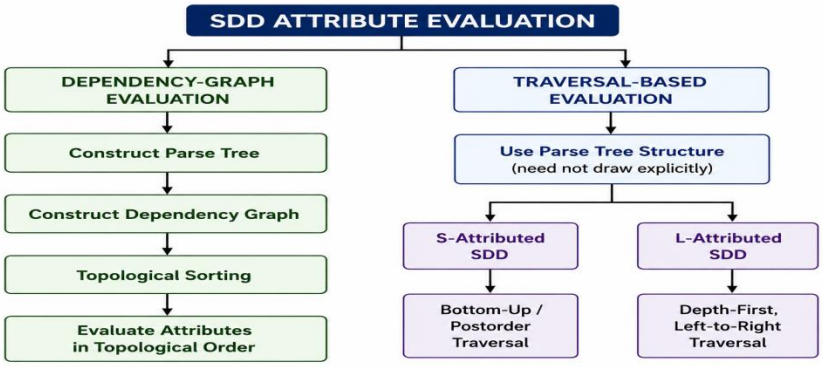
`T.type` → **synthesized** `L.in` → **inherited** `id.type` → **inherited/synthesized**

S-attributed SDD ⊂ L-attributed SDD. Every S-attributed SDD is also L-attributed, but every L-attributed SDD is not necessarily S-attributed.

S-attributed SDD	L-attributed SDD
1. Uses only synthesized attributes.	1. Can use synthesized and inherited attributes.
2. Information flows mainly from child to parent.	2. Information can flow from parent/siblings to the child, subject to restrictions.
3. It is a special case of L-attributed SDD.	3. It is a more general class than S-attributed SDD.
4. Does not allow inherited attributes.	4. Allows restricted inherited attributes.
5. Evaluation is naturally bottom-up.	5. Evaluation can be done left-to-right.
6. Convenient for bottom-up parsers, such as LR parsers.	6. Convenient for top-down parsers, such as LL parsers.
7. Example: E→E+T `{E.val = E1.val + T.val}`	7. Example: D→TL `{L.in = T.type }`
8. Simpler to implement.	8. More complex than S-attributed SDD.

➢ Evaluation Orders for SDD's

Evaluation order specifies the order in which the attributes of an SDD are evaluated. The order must satisfy the dependencies among the attributes



In Syntax-Directed Definitions (SDD), evaluation is mainly done in 2 ways, based on how the attributes are evaluated:

First construct a parse tree to implement any of these methods

1. Dependency Graph / Topological Sorting

   Construct a dependency graph showing dependencies between attribute instances.

   An edge a → b means attribute b depends on a.

   Evaluate attributes in topological order.

   Applicable to a general acyclic SDD.

2. Traversal-Based Evaluation

   Attributes are evaluated by traversing the parse tree in a particular order.

This is commonly divided into:

a) S-attributed SDD → evaluated using bottom-up / postorder traversal because it uses only synthesized attributes. i.e. Children → Parent
This makes S-attributed SDDs particularly suitable for bottom-up parsing.
b) L-attributed SDD → evaluated using depth-first, left-to-right traversal, because inherited attributes have restricted dependencies. Typically: Parent → Left child → Right child

Dependency Graph:

A directed graph showing the flow of information between attribute instances.

For each attribute in the parse tree, create a node in the dependency graph.

Add an edge from node A to node B if the semantic rule defining attribute B requires the value of attribute A.

Nodes/vertices = Attribute of symbol

Edges = dependecies between attributes

Example : A → B C A.x = B.x + C.x

Therefore, B.x and C.x must be evaluated before A.x.

Topological Sort:

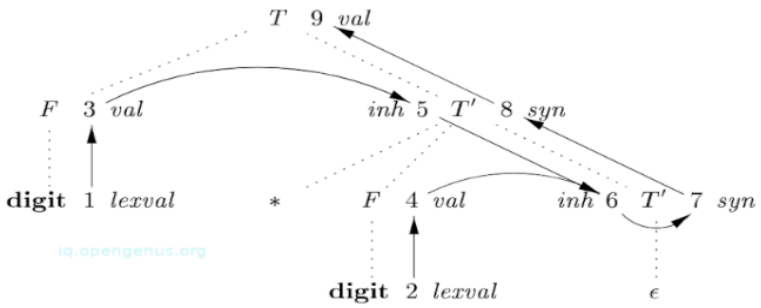
Topological ordering is an ordering of the nodes of a dependency graph such that every node appears after all the nodes on which it depends.

A topological sort produces a linear ordering of the nodes (attributes).

The only valid orders are those where, for every directed edge from node X to node Y, X appears before Y in the sequence.

This method can be used for a general SDD when the dependency graph is **acyclic (no cycle)**. If a cycle is found during the topological sort (e.g., the algorithm cannot find a node with no incoming edges after removing nodes), then there is no valid evaluation order, and theSDD is not well-defined for that parse tree.

A topological order is not necessarily unique. There can be more than one valid topological order.



One topological sort is the order in which the nodes have already been numbered: 1,2,... ,9. • There are other topological sorts as well, such as 1,3,5,2,4,6,7,8,9.

➢ Annotated Parse Tree

The parse tree containing the values of attributes at each node for given input string is called annotated or decorated parse tree

The process of computing and assigning attribute values to the nodes is called annotating or decorating the parse tree.

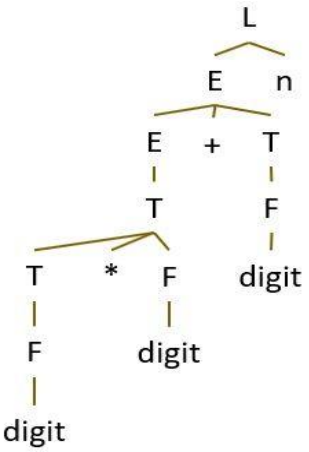
Synthesized attributes are typically evaluated in a bottom-up fashion, while inherited attributes are evaluated top-down. In cases where both types of attributes exist, a dependency graph may be used to determine the correct evaluation order.

1. **Annotated parse trees are about how the input is parsed and annotated**, giving a step-by-step visualization of semantic actions during parsing.
2. **Dependency graphs are about how attribute values rely on each other**, helping to order computations and identify circular dependencies.

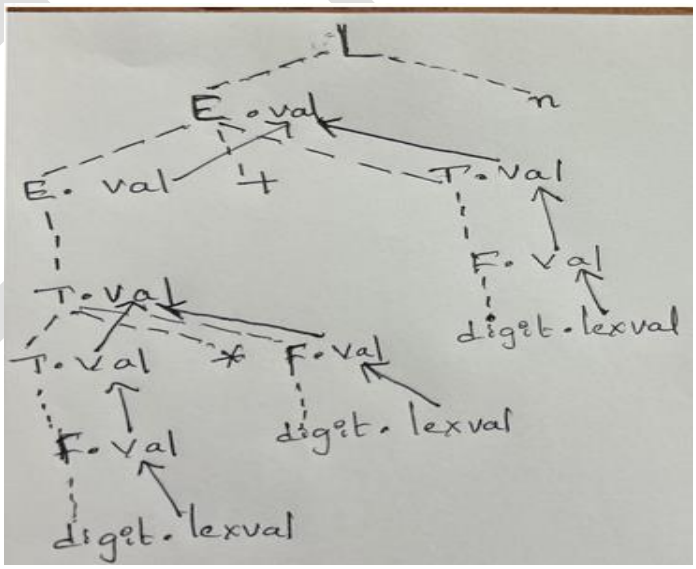
Example : Annotated parse tree for 3*5+4 (S-Attributed SDD)

SDD for simple desk calculator

	PRODUCTION	SEMANTIC RULES
1)	$L \rightarrow E$ **n**	$L.val = E.val$
2)	$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
3)	$E \rightarrow T$	$E.val = T.val$
4)	$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
5)	$T \rightarrow F$	$T.val = F.val$
6)	$F \rightarrow (E)$	$F.val = E.val$
7)	$F \rightarrow$ **digit**	$F.val =$ **digit**.lexval

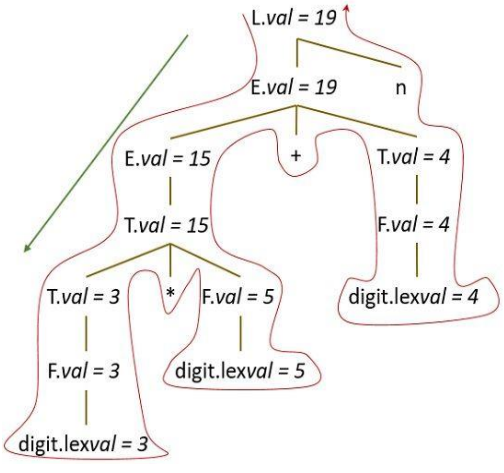


Parse Tree for Input String 3 * 5 + 4 n

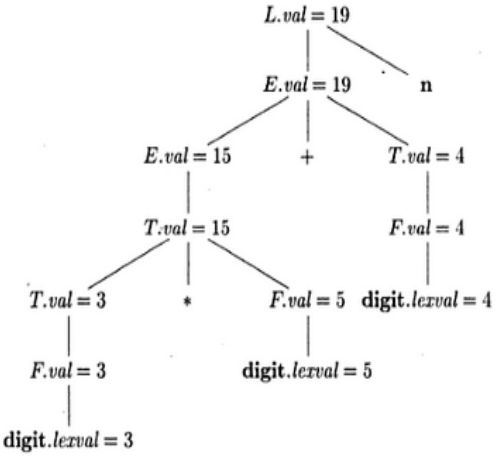


Dependency Graph

Bottom-up traversing



Annotated Parse Tree



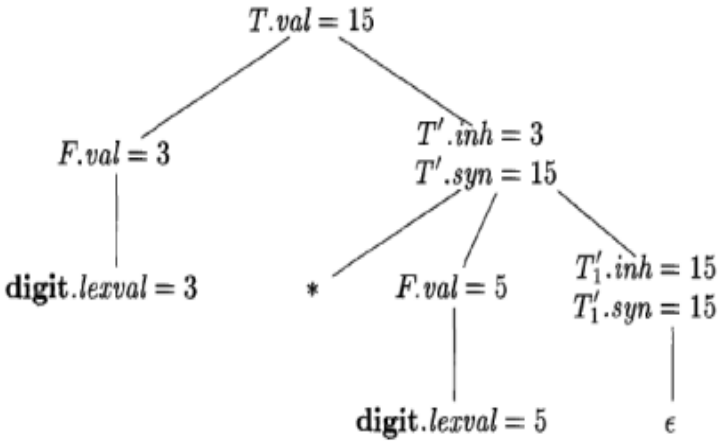
Example : Annotated parse tree , Dependency graph for 3*5 (Top-down)
(L-Attributed SDD)
CFG

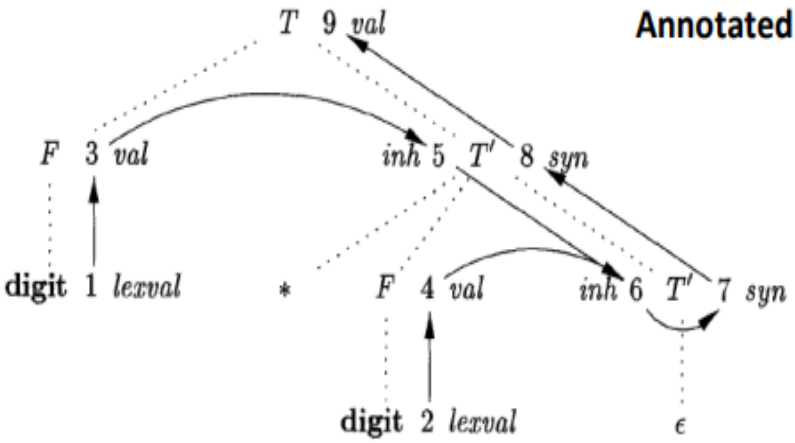
T →T*F | F
F → digit

Left recursion is no allowed for Top down parsing. So after elimination of left recursion the G will be

T → FT'
T' → *FT' | €
F→ digit

	PRODUCTION	SEMANTIC RULES
1)	$T \rightarrow F\,T'$	$T'.inh = F.val$ $T.val = T'.syn$
2)	$T' \rightarrow *\,F\,T_1'$	$T_1'.inh = T'.inh \times F.val$ $T'.syn = T_1'.syn$
3)	$T' \rightarrow \epsilon$	$T'.syn = T'.inh$
4)	$F \rightarrow \mathbf{digit}$	$F.val = \mathbf{digit}.lexval$





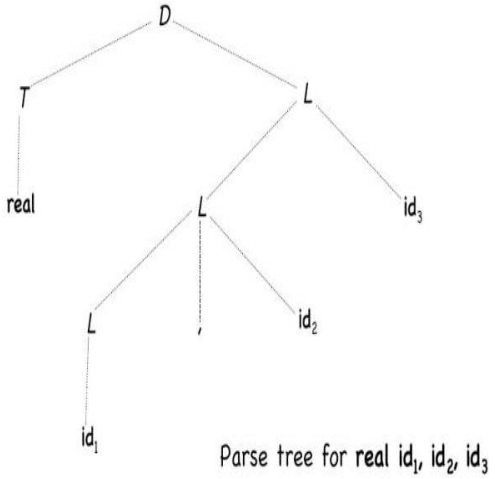
Annotated parse tree for 3 * 5

Dependency Graph Dotted line represents parse tree

By following topological order 1 2 3 4 5 6 7 8 9

Example : Annotated parse tree , Dependency graph for **Type Declaration (L-Attributed SDD)**

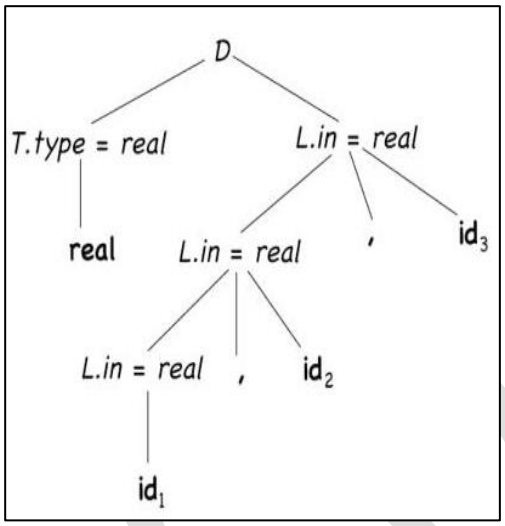
D -> T L	L.inh = T.type
T -> int	T.type = integer
T -> float	T.type = float
L -> L1 , id	L1.inh = L.inh; addType(id.entry, L.inh)
L -> id	addType(id.entry, L.inh)



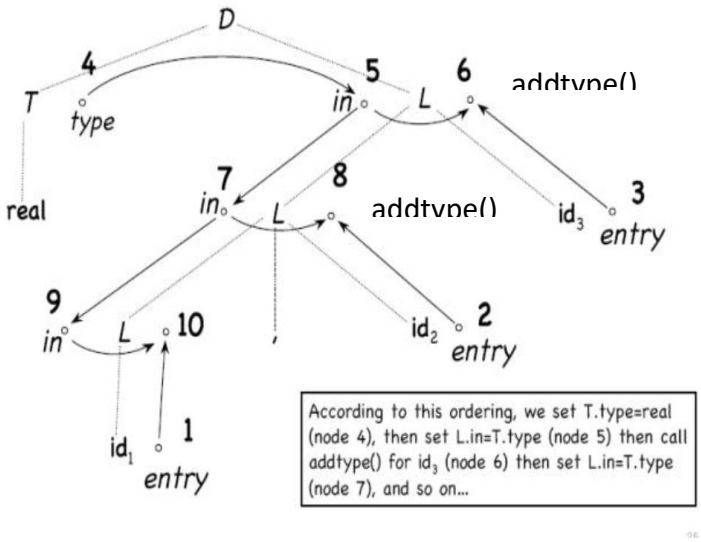
Parse tree for real id_1, id_2, id_3

- Symbol T is associated with a synthesized attribute *type*.
- Symbol L is associated with an inherited attribute *in*.

Annotated Parse tree for real id1,id2, id3



Dependency graph



To evaluate the L-attributed SDD, start evaluating the following order:

1. The value of the inherited attribute.
2. Then the value of synthesizing attributes

➢ Syntax-Directed Translation Schemes

• Syntax-directed translation schemes are a complementary notation to syntax directed definitions.

• All of the applications of syntax-directed definitions can be implemented using syntax-directed translation schemes.

A *translation scheme* is a context-free grammar whereby semantic rules are embedded within the right sides of productions.

• The program fragments are called semantic actions and can appear at any position within a production body.

When we construct a parse tree for a translation scheme, we use an extra child that is connected by a dashed line to the node of the production to indicate an action.

In the parse tree, the order in which actions appear is the order in which they are executed.

Types of SDT Schemes

Syntax-Directed Translation (SDT) schemes are classified based on the placement of semantic actions within a grammar production

1. Postfix (or Suffix) Scheme - Action at the end

   Actions are placed **after** the symbols in the production. Corresponds naturally with **bottom-up parsing**.

   Example: E → E + T { print("+"); }

   - Attributes are typically synthesized and computed after processing children.
   - Executed immediately when a reduction occurs in bottom-up parsing.
   - Use **postorder** traversal (bottom-up evaluation)

2. Prefix Scheme - Action at the beginning of production RHS

   Actions are placed **before** symbols in the production.

   Aligns with **top-down parsing**.

   Example: `E → { print("+"); } E + T`

   Used when context or inherited attributes need to be computed earlier.

3. Embedded or Mixed Actions Scheme

   Actions can be inserted anywhere within the production rule.

   Offers flexibility but requires careful ordering to ensure dependencies are satisfied.

   If something must be done after one symbol but before the next, place the action in between them.

   Example: E → E { print("+"); }

For actions at the beginning or middle, follow a **depth-first**, left-to-right traversal, executing each semantic action when its position is reached.

General Steps to Implement an SDT

Step 1: Ignore the Semantic Actions

Ignore all the semantic actions in the productions. Parse the input using the underlying context-free grammar (CFG) and construct the parse tree.

Step 2: Add Nodes for Semantic Actions

Add a separate child node for every semantic action at its corresponding position in the parse tree.

The children of each node should appear from left to right in the same order as the grammar symbols and semantic actions in the RHS.

Step 3: Perform Preorder Traversal

Perform a preorder traversal of the modified parse tree, visiting the nodes from left to right.

Parent → Left child → Right child

Step 4: Execute the Semantic Actions

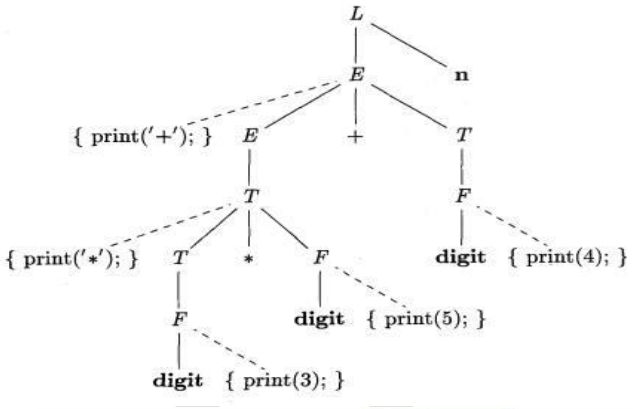
Whenever a node representing a semantic action is encountered during the traversal, execute that action.

Example : INFIX TO PREFIX

The parse tree for expression 3 * 5 + 4 with actions inserted. If we visit the nodes in *preorder*, we get the prefix form of the expression: + * 3 5 4. The following is a parse tree for an expression *3 * 5 + 4*;

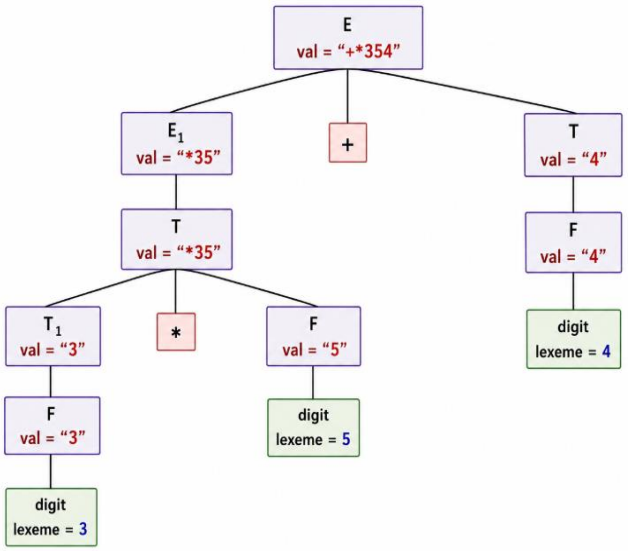
i) SDT INFIX TO PREFIX

L → En
E → { print('+'); } E₁ + T
E → { print('-'); } E₁ − T
E → T
T → { print('*'); } T₁ * F
T → { print('/'); } T₁ / F
T → F
F → (E)
F → digit { print(digit.lexval); }



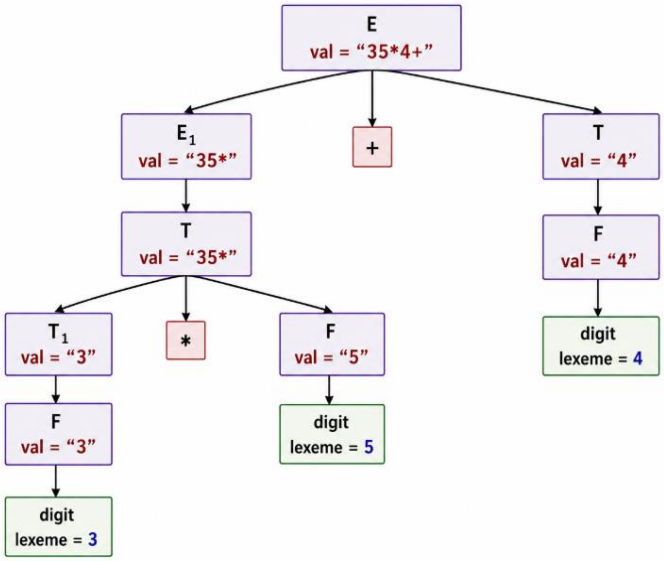
ii) SDD INFIX TO PREFIX

Production	Semantic Rule
E → E₁ + T	`E.val = '+' || E₁.val || T.val`
E → E₁ – T	`E.val = '-' || E₁.val || T.val`
E → T	`E.val = T.val`
T → T₁ * F	`T.val = '*' || T₁.val || F.val`
T → T₁ / F	`T.val = '/' || T₁.val || F.val`
T → F	`T.val = F.val`
F → (E)	`F.val = E.val`
F → digit	`F.val = digit.lexeme`



SDD INFIX TO POSTFIX

Production	Semantic Rule
E → E₁ + T	`E.val = E₁.val || T.val || '+'`
E → E₁ – T	`E.val = E₁.val || T.val || '-'`
E → T	`E.val = T.val`
T → T₁ * F	`T.val = T₁.val || F.val || '*'`
T → T₁ / F	`T.val = T₁.val || F.val || '/'`
T → F	`T.val = F.val`
F → (E)	`F.val = E.val`
F → digit	`F.val = digit.lexeme`



.'||' denotes concatenation

SDT INFIX TO **POSTFIX**

$E \rightarrow E_1 + T$ { print('+'); }

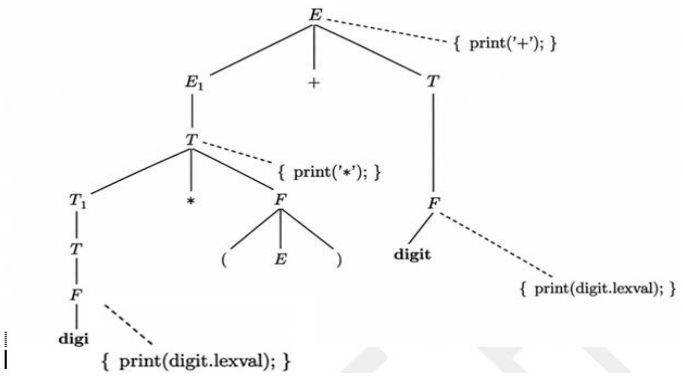
$E \rightarrow T$

$T \rightarrow T_1 * F$ { print('*'); }

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow$ digit { print(digit.lexval); }



Semantic Rules with Controlled Side Effects

The following methods will be used to control side effects in SDDs:

- Allow for unintentional side effects that do not affect attribute evaluation. Allow side effects when attribute evaluation based on any topological sort of the dependency network results in a "correct" translation, where "correct" is determined by the application.
- Limit the evaluation orders that can be used so that each order that can be utilized produces the exact translation. The constraints might be viewed as hidden edges in the dependency graph.

➢ Applications of SDD / SDT

- Executing Arithmetic Expressions
- Conversion from infix to postfix expression
- Conversion from infix to prefix expression
- For Binary to decimal conversion
- Counting the number of Reductions
- Creating a Syntax tree
- Generating intermediate code
- Storing information into the symbol table
- Type checking

➢ Construction of Syntax Trees

Syntax trees (or abstract syntax trees) are intermediate representations of a program's structure in a compiler, constructed from parse trees by using syntax-directed definitions to represent operators as internal nodes and operands as leaf nodes, and then using semantic actions to link these components to form the desired tree structure.

Each leaf node represents an operand and each inner node represents an operator in a syntax tree.



- Leaf nodes for operators and keywords are removed.
- Internal nodes corresponding to uninformative non-terminals are replaced by the more meaningful operators.

Rules for Constructing Syntax Tree

Production	Semantic Action

$E \rightarrow E_1 + T$	{ E.nptr = mknode(E_1.nptr, '+', T.nptr); }
$E \rightarrow T$	{ E.nptr = T.nptr; }
$T \rightarrow T_1 * F$	{ T.nptr = mknode(T_1.nptr, '*', F.nptr); }
$T \rightarrow F$	{ T.nptr = F.nptr; }
$F \rightarrow id$	{ F.nptr = mknode(null, id.name, null); }

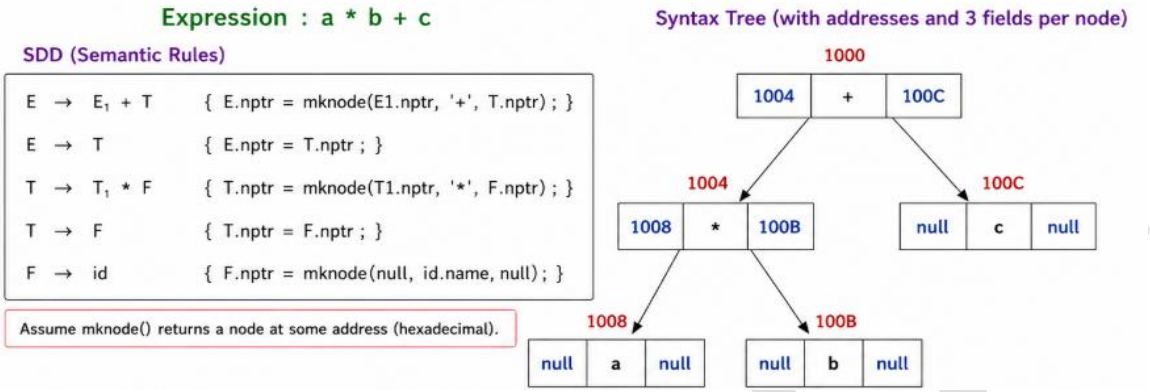
The syntax tree nodes can all be treated as data with several fields. The operator is identified by one node element, whereas the remaining areas include a pointer to the operand nodes. The node's label is also known as the operator.

`mknode(left child, operator/value, right child)`

It creates an operator node with the name op and two fields, containing left and right pointers.

`nptr` → pointer/reference to the syntax-tree node.

So, this is exactly an example of an **S-attributed SDD used for syntax tree construction** because all the attributes are synthesized from the children.



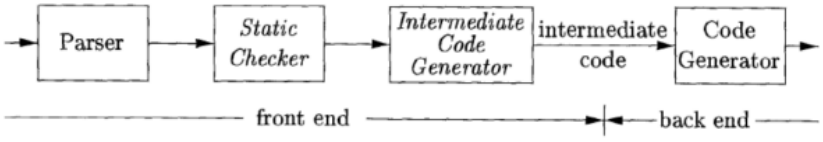
If both the left child pointer and right child pointer are null, it means that the node has no children. Therefore, it is a leaf node (terminal node) in the syntax tree.

SDD vs SDT

Aspect	SDD (Syntax-Directed Definition)	SDT (Syntax-Directed Translation)
Definition	It's a formal way to define attributes (synthesized or inherited) and semantic rules for grammar productions.	It's a method of translating input by associating actions with grammar productions during parsing.
Focus	Specifies **what** information should be computed and how attributes are related.	Specifies **when and how** computations/actions are performed during parsing.
Purpose	Used to describe semantic rules and attribute evaluation strategies abstractly.	Used to implement translation by embedding actions (like code generation) within grammar rules.
Components	Attributes (inherited or synthesized), semantic rules, and dependency graphs.	Action code (e.g., print statements, code generation), inserted at specific points in grammar.
Execution	The attribute values are computed after parsing using dependency graphs or evaluation orders.	Actions are executed immediately during parsing at specified positions (before or after symbols).
Relation	Forms the theoretical foundation for semantics in grammars.	Is a practical implementation technique based on SDD.
Example	Defining E.val = E1.val + T.val for E → E1 + T.	Writing E → E1 + T { print('+') } where the print action is executed during parsing.
Type of description	Declarative – specifies relationships.	Imperative – specifies computation steps and side-effects.
Use cases	Used to define expressions, type checking, etc.	Used for generating intermediate code, machine code, etc.

INTERMEDIATE–CODE GENERATION

Intermediate Code Generation (ICG) in compiler design is the process of converting high-level source code into an intermediate representation (IR). This step improves portability and efficiency, acting as a bridge between source code and machine code. The IR is independent of machine architecture, facilitating optimization and easier translation into target machine code across different platforms.



Intermediate representation

Intermediate code can be represented in two ways:

1. High Level intermediate code

A compiler or an interpreter generates high-level intermediate code, which stands between the source code and the machine code. It is portable and platform-independent, allowing it to run on various platforms.

2. Low Level intermediate code

Low-level intermediate code is a type of code that sits between high-level programming languages and machine code. It's typically written in assembly language or another low-level language. Low-level intermediate code is often used to develop operating systems, device drivers, and other software



Intermediate Code Generator: Translates the source code into an intermediate representation (intermediate code), which is a machine-independent form.

Back End: This part of the compiler focuses on generating the final machine code.

- *Code Generator*: Takes the intermediate code and converts it into the target machine code, which can be executed by the computer.

The intermediate code acts as a bridge between the front end and the back end, allowing for optimizations and machine independence.

❖ *Advantages of Intermediate Code*

- *Portability*: Intermediate code is platform-independent, making it easier to generate machine code for multiple architectures without rewriting the entire compiler.
- *Optimization*: Intermediate code provides an abstract, simpler representation, allowing the compiler to perform optimizations before generating machine-specific code.
- *Simplified Compilation Process*: It divides the compilation into manageable phases, separating concerns between front-end syntax processing and back-end code generation.
- *Reusability*: The intermediate code can be reused for different target machines, simplifying compiler design and maintenance.
- *Error Detection*: It helps in early detection of errors, as transformations in intermediate code can reveal inconsistencies before reaching machine code generation.

❖ *Disadvantages of Intermediate Code Generation*

- *Increased Compilation Time*: Generating and processing intermediate code adds extra steps, slightly lengthening the overall compilation time.
- *Memory Overhead*: Storing and manipulating the intermediate code requires additional memory resources during the compilation process.
- *Loss of Source Code Context*: Some high-level details may be abstracted away in the intermediate code, making certain optimizations or error handling more challenging.
- *Complexity*: Designing an effective intermediate representation adds complexity to the compiler's architecture.

➢ Intermediate code Representation

Intermediate Code Representation (ICR) is a machine-independent representation of a source program generated by the compiler between the **front end** and **back end**. It makes the program easier to optimize and translate into target machine code.
It can be represented mainly in **two forms**:
It can be represented in

1) Linear form

   In linear form, the intermediate code is represented as a sequence of instructions or statements.

   a. Post fix notation
   b. Three Address code (TAC)
   c. Static Single Assignment (SSA)

2) Tree form

   In tree form, the intermediate representation shows the hierarchical structure of expressions.

   a. Syntax tree
   b. Directed acyclic graph (DAG)

1. Linear form

a) Postfix Notation

Operators are written after their operands. It does not require parentheses.

Example 1 : a + b * c
Postfix : a b c * +

Example 2 : P + (–Q + R * S)
Postfix notation PQ - RS * ++ T

b) Three-Address Code (TAC)

A sequence of simple statements, where each statement contains at most three addresses.

Three-address code (TAC) is a compiler intermediate representation where each instruction typically involves three operands: two inputs and one output. It's used to simplify code optimization and translation into machine code.

Variables *can be assigned multiple times.* Looks like a simplified machine code with temporaries.

Three-address code, often known as TAC or 3AC

*Example – The three address code for the expression a + b * c + d:*

T 1 = b * c

T 2 = a + T 1

T 3 = T 2 + d T 1 , T 2 , T 3 are temporary variables.

c) Static Single Assignment (SSA)

Each variable is assigned a value only once.

Different versions of a variable are used for subsequent assignments.

Special ϕ (phi) functions are introduced at control flow joins to select the correct variable version.

But in control flow (like if-else or loops), the same variable can be defined in multiple paths. At the join point of these paths, we need a way to say: "Take the value depending on which path was executed." That's where the ϕ-function comes in.

Example : a = b + c; d = a * 2; a = d - 1;

TAC	SSA
t1 = b + c a = t1 t2 = a * 2 d = t2 t3 = d - 1 a = t3 'a' is assigned twice.	a1 = b + c t1 = a1 * 2 d1 = t1 a2 = d1 - 1

Example :

```
if (cond)
   x = 1;
else
   x = 2;
y = x + 3;
```

TAC	SSA
if cond goto L1 x = 2 goto L2 L1: x = 1 L2: y = x + 3	if cond goto L1 x1 = 2 goto L2 L1: x2 = 1 L2: x3 = ϕ(x1, x2) y1 = x3 + 3

2. Tree Form

a) Syntax tree

Represents the syntactic structure of a program or expression as a tree.

Operators are usually internal nodes and operands are leaf nodes. Least priority operator is root node

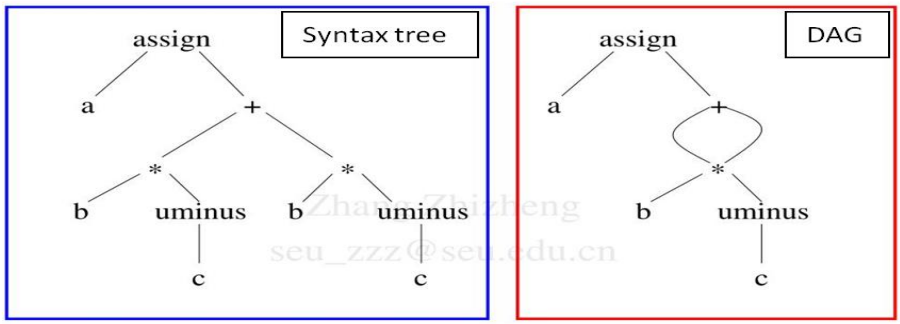
b) Directed Acyclic Graph (DAG)

Similar to a syntax tree, but common subexpressions are shared instead of being repeated.

Helps in detecting common subexpressions and performing code optimization.

A DAG is a graph, not strictly a tree, because a node may have more than one parent.

a:=b*-c+b*-c



➢ Variants of Syntax Trees

In compiler design, syntax trees can be represented in different ways, with the most common variants being

1. Directed Acyclic Graphs (DAGs) for expressions
2. Value-Number Method for constructing DAGs.

1. Directed Acyclic Graph (DAG) for Expressions:

- A DAG is a variation of the syntax tree that efficiently represents common sub-expressions within an expression.
- It identifies and collapses repeated sub-expressions, creating a more compact representation compared to a standard syntax tree.
- In a DAG, if a sub-expression appears multiple times, it will be represented by a single node with multiple incoming edges (representing the different places it's used).
- This structure helps compilers optimize code generation by recognizing and reusing computations for common sub-expressions.

Following rules are used for the construction of DAGs-

Rule-01: In a DAG,

- Interior nodes always represent the operators.
- Exterior nodes (leaf nodes) always represent the names, identifiers or constants.

Rule-02: While constructing a DAG,

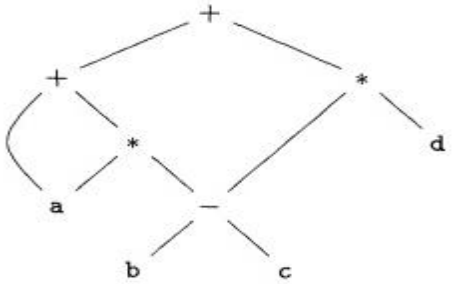
- A check is made to find if there exists any node with the same value.
- A new node is created only when there does not exist any node with the same value.
- This action helps in detecting the common sub-expressions and avoiding the re-computation of the same.

Rule-03:

The assignment instructions of the form x:=y are not performed unless they are necessary.

- Example

In the expression a + a * (b - c) + (b - c) * d, a DAG would represent (b - c) only once, even though it's used twice in the expression.

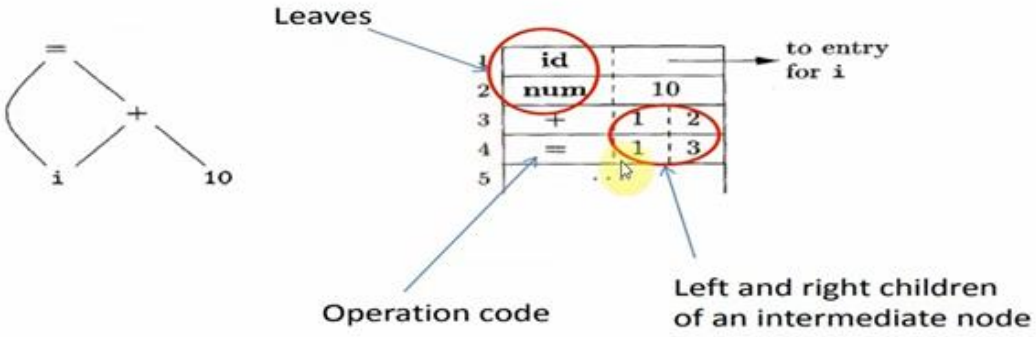


2. Value-Number Method for Constructing DAGs:

- Nodes of a syntax tree or DAG are stored in an ***array of records***.
- The **integer index** of the record for a node in the array is known as the **value number** of that node.
- Each row of the array represents one record, and therefore one node.
- In each record, the **first field** is an **operation code**, indicating the **label of the node**.

Leaves	**One** additional field	**Holds the lexical value** (either a symbol-table pointer or a constant
Interior nodes	**Two** additional fields	Indicating the **left and right children**

- **Example**: **Nodes of a DAG for $i = i + 10$ allocated in an array**

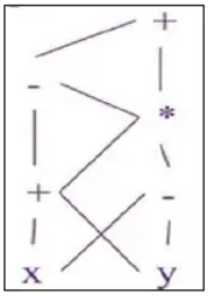


- The node labeled "**+**" has value number **3**, and its **left** and **right children** have value numbers **1** and **2**

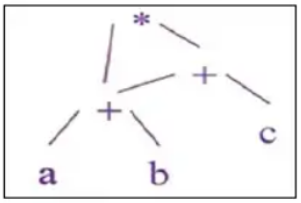
Examples for DAG

Construct the DAG for the expression

((x+y)-((x+y)*(x-y)))+((x+y)*(x-y))



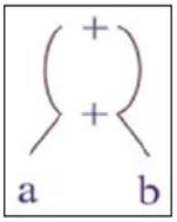
(a+b)*(a+b+c)



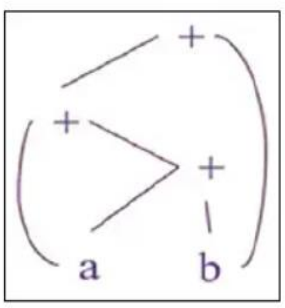
Construct the DAG and identify the value numbers for the subexpressions of the following expressions, assuming + associates from the left.

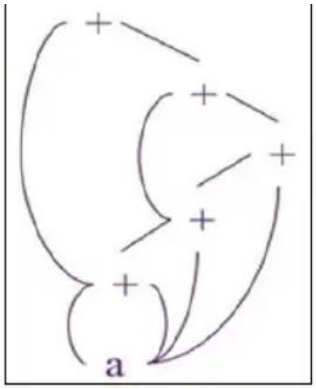
a + a + ((a+ a + a + (a + a + a + a))

a + b+ (a + b)



a + b + a + b





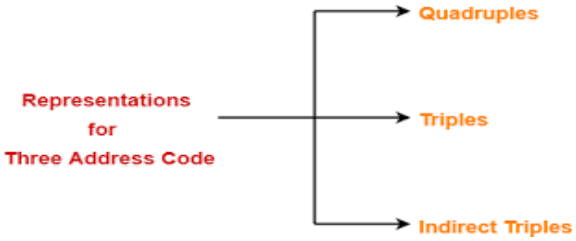
➢ Common Three Address Instruction form

Three-Address Statements are similar to assembly code. We will consider the following statements.

1. Assignments.
   Two possible forms:
   - x:= y op z where op is a binary arithmetic or logical operation,
   - x:= op y where op is a unary operation (minus, negation, conversion operator)
2. Copy statements : They have the form x = y.
3. Unconditional jumps They have the form
   goto L where L is a symbolic label of a statement.
4. Conditional jumps.
   They have the form
   if x relop y goto L
   where statement L is executed if x and y are in relation relop.
5. Procedure calls.
   They have the form
   param x1
   param x2
   :
   :
   param xn
   call p, n
   corresponding to the procdure call p(x1, x2, ..., xn)
   Return statement return y
   where y representing a returned value is optional.
6. Indexed assignments.
   They have the form x := y[i] or x[i] := y.
7. Address assignments.
   They have the form x := &y which sets x to the location of y.
8. Pointer assignments.
   They have the form
   - x := *y where y is a pointer and which sets x to the value pointed to by y
   - *x := y which changes the location of the value pointed to by x.

➢ Implementation of Three Address Code / Data structures of 3AC / Types of TAC representation

There are three different ways to express three address codes:



CD-IT

a) Quadruple

It is a structure that has four fields: op, arg1, arg2, and result. The operator is denoted by op, arg1, and arg2 denote the operands, and the result is used to record the outcome of the expression.

These quadruples play a crucial role in breaking down high-level language statements into more digestible parts, facilitating compilation-stage analysis and optimization procedures.

Benefits of Quadruple

- For global optimization, it's simple to restructure code.
  Using a symbol table, one may rapidly get the value of temporary variables.

Drawbacks of Quadrule

There are a lot of temporary items.

The establishment of temporary variables adds to the time and space complexity.

Example : Convert a = -b * c + d into three address codes .

The following is the three-address code:

$t_1 = -b$

$t_2 = c + d$

$t_3 = t_1 * t_2$

$a = t_3$

Quadruples are used to symbolize these statements:

#	Op	Arg1	Arg2	Result
(0)	unimus	b	-	t_1
(1)	+	c	d	t_2
(2)	*	t_1	t_2	t_3
(3)	=	t_3	-	a

b) Triples

Instead of using an additional temporary variable to represent a single action, a pointer to the triple is utilized when a reference to another triple's value is required. As a result, it only has three fields: op, arg1, and arg2.

Benefits of Triples

- Triples make it easier to analyze and optimize code by disassembling difficult high-level language constructs into smaller, more manageable parts
  - Triples facilitate error, data flow, and control flow analysis of code, facilitating improved debugging and comprehension

Drawbacks of Triples

- Temporaries are difficult to rearrange since they are implicit.
- It's tough to optimize since it necessitates the relocation of intermediary code. When a triple is relocated, all triples that relate to it must likewise be changed. The symbol table entry can be accessed directly using the pointer.

Example

Convert a = -b * c + d into three address codes.The following is the three-address code:

$t_1 = -b$

$t_2 = c + d$

$t_3 = t_1 * t_2$

$a = t_3$

	Op	Arg1	Arg2
(10)	unimus	b	-
(11)	+	c	d
(12)	*	(10)	(11)
(13)	=	a	(12)

c) Indirect Triples

The indirect triple representation uses an extra array to list the pointers to the triples in the desired order. This is known as indirect triple representation.

Indirect Triple will be

Example

Convert a = -b * c + d into three address codes.

The following is the three-address code:

$t_1 = -b$

$t_2 = c + d$

$t_3 = t_1 * t_2$

$a = t_3$

	ptr
(0)	(10)
(1)	(11)
(2)	(12)
(3)	(13)

	Op	Arg1	Arg2
(10)	unimus	b	-
(11)	+	c	d
(12)	*	(10)	(11)
(13)	=	a	(12)

In this, it can only need to refer to a pointer (0), (1), (2), (3) which will further refer pointers (10), (11), (12), (13) respectively & then pointers (11), (12), (13) point to triples that is why this representation is called indirect triple representation.

Example: Translate the expression X = - (a + b) * (c + d) + (a + b + c) into quadruples and triples and indirect triples.

Example

X = - (a + b) * (c + d) + (a + b + c)

Three address code

t1 = a + b
t2 = minus t1
t3 = c + d
t4 = t2 * t3
t5 = t1 + c
t6 = t4 + t5
X = t6

Quadruples

	op	arg1	arg2	result
0	+	a	b	t1
1	uminus	t1	–	t2
2	+	c	d	t3
3	*	t2	t3	t4
4	+	t1	c	t5
5	+	t4	t5	t6
6	=	t6	–	X

Triples

	op	arg1	arg2
0	+	a	b
1	uminus	(0)	–
2	+	c	d
3	*	(1)	(2)
4	+	(0)	c
5	+	(3)	(4)
6	=	(5)	X

Indirect Triples

	ptr	op	arg1	arg2
0	(0)	+	a	b
1	(1)	uminus	(0)	–
2	(2)	+	c	d
3	(3)	*	(1)	(2)
4	(4)	+	(0)	c
5	(5)	+	(3)	(4)
6	(6)	=	(5)	X

Example

- b * minus c + b * minus c

Three address code

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5

Quadruples

	op	arg1	arg2	result
	minus	c		t1
	*	b	t1	t2
	minus	c		t3
	*	b	t3	t4
	+	t2	t4	t5
	=	t5		a

Triples

	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

Indirect Triples

	op
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)

	op	arg1	arg2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

3AC vs SSA Single static address

Feature	TAC (Three Address Code)	SSA (Static Single Assignment)
Assignment of variables	A variable can be assigned multiple times	Each variable is assigned exactly once
Variable names	Same variable name may reappear on LHS	New version of variable created for each assignment
Optimization	Supports optimization	Makes many optimizations easier and more efficient
ϕ (Phi) function	Not used	Used at control-flow merge points
Complexity	Simpler to generate	More complex than TAC
Relation	Basic intermediate representation	Refined form of TAC (VTUCS)

➢ **TAC for 1-D array**

EX1 x = A[i];

TAC

```
t1 = i * width                  // width = size of each element (e.g., 4 for int)
t2 = base(A) + t1
x  = *(t2)                      // load from memory
```

- width = number of bytes of one element.
- base(A) = starting address of array A.
- *(t) means indirect access (value at address t).

Ex2 A[i] = x;

TAC

```
t1 = i * width
t2 = base(A) + t1
*(t2) = x       // store into memory
```

a) Write TAC for the following

if A < B then 1 else 0

```
if (A < B) goto L1
T1 = 0
goto L2
L1:      T1 = 1
L2:
```

b) Write TAC for the following

```
for ( i=0; i<10 ; i++)
{
x = 4* I;
}
```

```
i = 0
L1: if i >= 10 goto L2
   x = 4 * i
   i = i + 1
   goto L1
L2:
```

Note: x= 4*I is TAC form. So, it can be with out temporary variables also

Write TAC for the following

```
i = 0;
while (i < 10)
{
   x = 4 * i;
   i++;
}
```

```
i = 0
L1:     if i < 10 goto L2
           goto L3
L2:     T1 = 4 * i
           x = T1
           T2 = i + 1
           i = T2
           goto L1
L3:
```

Write Three Address Code for the following expression-

```
i=0
do {
      x = 4*i;
      i++;
 }while (i<10);
```

```
(1) i=0
(2) T1 = 4*1
(3) x = T1
(4) T2 = i+1
(5) i = T2
(6) if  i<10  goto (2)
(7)
```

TAC for Switch Statement

```
switch (x) {
   case 1:  S1; break;
   case 2:  S2; break;
   case 3:  S3; break;
   default: S4;
}
```

```
if x == 1 goto L1
if x == 2 goto L2
if x == 3 goto L3
goto L4
L1: S1
   goto L5
L2: S2
   goto L5
L3: S3
   goto L5

L4: S4
L5:
```

The `goto L5` after each case is because of the **`break`** statement. It takes control directly to the end of the `switch`.

Example **TAC for Main calling a procedure**

```
int main() {
   int r;
   r = add(10, 23);
   print(r);
}
```

```
param 10
param 23
t1 = call add, 2
r = t1
param r
call print, 1
```

TAC for Procedure with return

```
int add(int a, int b) {
   return a + b;
}
```

```
proc add begin
   t1 = a + b
   return t1
proc add end
```

➢ TYPES AND DECLARATIONS

In compiler design, types and declarations define the structure and meaning of data within a programming language.

In compiler design, types and declarations define the structure and meaning of data within a programming language.

Applications of Types and Declarations

Applications of types and declarations are:

(1) Type checking (2) memory allocation (3) symbol table management

(4) scope management (5) type conversion (6) error detection

(7) function parameter checking (8) storage layout

(9) code generation (10) code optimization.

A type tells the compiler:

what kind of value a variable can store,

how much memory may be required,

what operations are allowed on that value.

Declarations introduce variables, functions, and other entities, specifying their types and other attributes.

Declaration involves allocating space in memory and entering type and name in the symbol table.

Type Expressions

A type expression is a notation used by a compiler to represent the type of a variable, expression, function, array, pointer, etc.

Type expression describes the structure of a data type using basic types and type constructors.

Declaration	Type Expression
int x	int
float x	float
int A[10]	array(10,int)
int *p	pointer(int)
int A[5][10]	array(5,array(10,int))
int f(int,float)	int × float → int

A basic type is a type expression. Typical basic types for a language include *Boolean, char, integer, float,* and *void;* the latter denotes "the absence of a value."

Concept	Meaning
Type name	The actual **name used in code** to refer to a type (int, float, Point, uint).
Type expression	The **internal representation** of a type used by the compiler, e.g., array(10, int) or pointer(float). It contains full information about structure, size, and nested types.

Example :

float A[10]; // Type name: float

Type name → float

Type expression → array(10, float)

- **Type Equivalence**

Type equivalence in compiler design refers to the rules used to determine whether two type expressions represent the same type.

This is a crucial aspect of type checking, where the compiler ensures that operations are performed on compatible data types. There are two primary approaches to type equivalence:

"If two type expressions are equal, return a specified type else error," says a common type-checking rule.

i) Name Equivalence

Two variables are equivalent if they are declared using the same type name.

Example:

typedef int Age; Age a;

typedef int Year; Year y;

- Here, even though Age and Year are both int, under strict *name equivalence*, *a* and *y* are considered *different* types.

ii) Structural Equivalence

Two types are equivalent *if their structures are the same*, regardless of the names used.

Example:

- struct { int x; float y; } A;
- struct { int x; float y; } B;

Under *structural equivalence*, A and B are the same type because their structure matches.

Note

It's not mandatory to check both.

.

- `Declarations`

When we encounter declarations, we need to lay out storage for the declared variables.

For every local name in a procedure, we create a Symbol Table(ST) entry including:

1. The type of the name
2. How much storage the name requires

$$
\begin{array}{lcl}
D & \rightarrow & T\ \mathbf{id}\ ;\ D \mid \epsilon \\
T & \rightarrow & B\ C \mid \mathbf{record}\ '\{'\ D\ '\}' \\
B & \rightarrow & \mathbf{int} \mid \mathbf{float} \\
C & \rightarrow & \epsilon \mid [\ \mathbf{num}\ ]\ C
\end{array}
$$

iq.opengenus.org

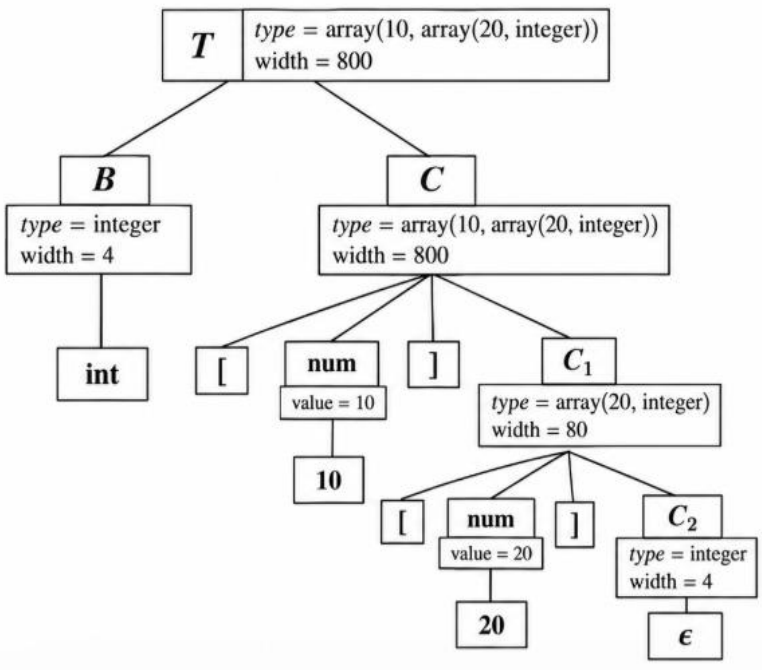
The fragment of the above grammar that deals with basic and array types

The non-terminal *D* generates a sequence of declarations. Non-terminal *T* generates basic types, array, or record types.
The non-terminal *B* generates a basic type either an *int* or *float*.
The non-terminal *C* generates strings of zero or more integers, each surrounded by brackets.
An array type consists of the basic type specified by *B* followed by array components specified by a non-terminal *C*. A record type is a sequence of declarations for fields of the record surrounded by curly braces.

- **Syntax-Directed Definition (SDD) for calculating** the **type and width (memory size) of declarations**

Syntax–Directed Definition (SDD) for Declarations and Types	
Production	**Semantic Rule(s)**
$D \rightarrow T$ **id** ; D	{ $enter(\textbf{id}.lexeme,\ T.type,\ T.width)$; } { $D.type = D_1.type;\quad D.width = D_1.width;$ }
$D \rightarrow \epsilon$	{ $D.type = null;\quad D.width = 0;$ }
$T \rightarrow B\ C$	{ $t = B.type;\quad w = B.width;$ } { $T.type = C.type;\quad T.width = C.width;$ }
$B \rightarrow$ **int**	{ $B.type = integer;\quad B.width = 4;$ }
$B \rightarrow$ **float**	{ $B.type = float;\quad B.width = 8;$ }
$C \rightarrow \epsilon$	{ $C.type = t;\quad C.width = w;$ }
$C \rightarrow$ [**num**] C_1	{ $C.type = array(\textbf{num}.value,\ C_1.type);$ $C.width = \textbf{num}.value \times C_1.width;$ }

Example: int a[10][20];



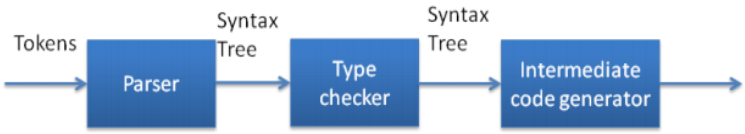
Leftmost derivation

$T \Rightarrow B\ C$
$\Rightarrow$ **int** C
$\Rightarrow$ **int** [**num**] C_1
$\Rightarrow$ **int** [10] C_1
$\Rightarrow$ **int** [10] [**num**] C_2
$\Rightarrow$ **int** [10] [20] C_2
$\Rightarrow$ **int** [10] [20] ϵ
$\Rightarrow$ **int** [10] [20]

Result for the example

Type : array(10, array(20, integer))
Width : 800 bytes

➢ Type Checking



Type checking in compiler design is the process of verifying that the types of operands in expressions and other language constructs are compatible according to the rules of the programming language.

It ensures that operations are performed on data of the correct type, preventing runtime errors and promoting program correctness

A compiler should ensure that the source program follows the source language's syntactic and semantic conversions.

Types of Type Checking

There are two kinds of type checking:

a) Static Type Checking

Static type checking is defined as type checking performed at compile time. It checks the type variables at compile-time, which means the type of the variable is known at the compile time. Type checking or static checking is performed by the compiler (checking is done at the compiler time).

Static Type-Checking is also used to determine the amount of memory needed to store the variable. Languages like Pascal and C have static type checking.

b) Dynamic Type Checking

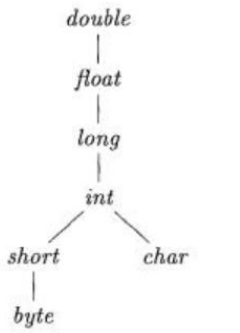
Dynamic Type Checking is defined as the type checking being done at run time. In Dynamic Type Checking, types are associated with values, not variables. Implementations of dynamically type-checked languages runtime objects are generally associated with each other through a type tag, which is a reference to a type containing its type information. Dynamic typing is more flexible.

➢ Type Conversions

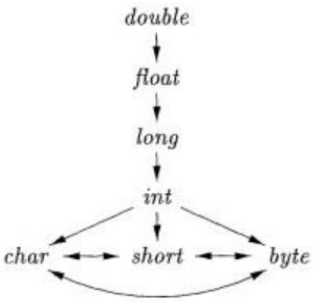
Type conversion rules vary from language to language.

Conversion from one type to another is said to be ***implicit*** if it is done automatically by the compiler. Implicit type conversions, also called *coercions,*

- Widening Conversions:
  These conversions preserve information and allow a type lower in a hierarchy to be converted to a higher type (e.g., char to int or float).
- Narrowing Conversions:
  These conversions can result in information loss and are defined by a graph where a types can be narrowed to t if a path exists from s to t.



(a) Widening conversions



(b) Narrowing conversions

Conversion is said to be explicit if the programmer must write something to cause the conversion. Explicit conversions are also called casts.

Expression	Conversion
`int + int`	None
`int + float`	`int → float`
`float + int`	`int → float`
`float + float`	None
`float` assigned to `int`	`float → int`

➢ SDT FOR ASSIGNMENT STATEMENTS TO GENERATE TAC

In the syntax directed translation, assignment statement is mainly deals with expressions. The expression can be of type real, integer, array and records

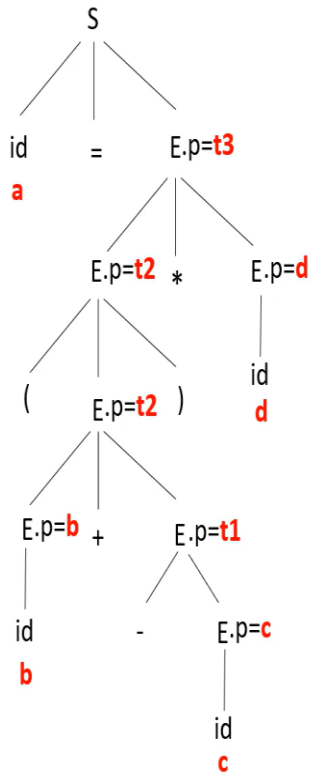
S → id = E

E → E + E | E * E | -E | (E) | id

- lookup(id.name): Finds the symbol table entry for the identifier id.
- `EMIT(...)`: Appends a new three-address instruction to the generated code.
- `E.place`: The temporary variable or identifier that holds the result of evaluating the expression `E`.
- A helper function, `newtemp()`, is used to generate a new unique temporary variable name for each intermediate result.

SDT for Assignment Statements

Production Rule	Semantic actions
S → id :=E	{p = lookup(id.name); If p ≠ nil then Emit (p = E.place) Else Error; }
E → E1 + E2	{E.place = newtemp(); Emit (E.place = E1.place '+' E2.place) }
E → E1 * E2	{E.place = newtemp(); Emit (E.place = E1.place '*' E2.place) }
E → -E1	{E.place = newtemp(); Emit (E.place = uminus E1.place}
E → (E1)	{E.place = E1.place}
E → id	{p = lookup(id.name); If p ≠ nil then E.place = p Else Error; }



Example: a = (b + -c)*d

Three Address Code:

t1 = uminus c
t2 = b + t1
t3 = t2 * d
a = t3

➢ Control Flow

Control flow dictates the order in which code is processed, including conditional execution (like if statements), loops, and function calls.

Control flow statements include **conditional statements**, **branching statements**, and **looping statements**. Control flow can be decided with the help of a boolean expression (an expression that evaluates to either true or false).

Boolean Expression:

A Boolean Expression is an expression that evaluates to either true or false.

The translation of boolean expressions is connected to the translation of statements like if-else statements and while statements. Boolean expressions are frequently used in programming languages

1. They are used for computing the logical values.
2. They are also used as conditional expression that alter the flow of control, such as if-then-else or while-do.

Boolean expressions are formed using Boolean operators. These operators, specifically AND, OR, and NOT, follow the C convention. They are applied to Boolean variables or relational expressions. Consider the grammar

E → E or E
E → E and E
E → not E
E → (E)
E → id relop id
E → TRUE
E → FALSE

The relop is any of <,≤,=, ≠, >, or ≥.. We assume that **or** and **and** are left associative. **or** has lowest precedence ,then **and** ,then **not**.

Methods of Translating Boolean Expressions:

When translating Boolean expressions into three-address code, we can use two different methods.

a) Numerical method.

Assign numerical values to true and false and evaluate the expression analogously to an arithmetic expression. This is convenient for boolean expressions which are not involved in flow of control constructs. • 1 is used to denote true. • 0 is used to denote false

It may generate unnecessary temporary variables and instructions, especially when the Boolean expression is used only as a condition.

b) Jump method or Control flow representation or Short Circuit Code

In compiler design, "short-circuit code" refers to a technique used in the evaluation of logical expressions, with operators AND (&&) and OR (||).

The main purpose of short-circuit evaluation is to optimize performance by avoiding unnecessary computations.

The Boolean expression is translated directly into conditional and unconditional jumps.

Boolean expression could generate three-address code without generating full code. The expression need not be evaluated in full. The compiler generates code that checks the first operand and only evaluates the second operand if necessary.

- AND → if first is FALSE → skip second , final result considered as FALSE
- OR → if first is TRUE → skip second, final result considered as TRUE
- NOT → interchange TRUE and FALSE labels

Benefits:

- Uses Usually fewer temporary variables
- - Efficiency: Reduces the number of evaluations, leading to faster execution.
- - Avoiding Side Effects: If the second operand has side effects (e.g., function calls that modify state), short-circuiting prevents those side effects from occurring when not needed

Example: if (a < b)

Numerical method	Jumping method
t1 = a < b if t1 goto L1 goto L2	if a < b goto L1 goto L2

Example : if (x<100|| x>200 && x!=y) x=0

Numerical method	Jumping method
t1 = x < 100 t2 = x > 200 t3 = x != y t4 = t2 && t3 t5 = t1 || t4 if t5 goto L1 goto L2 L1: x = 0 L2:	if x < 100 goto L1 goto L3 L3: if x > 200 goto L4 goto L2 L4: if x != y goto L1 goto L2 L1: x = 0 L2:

Example : if (a < b && c != d) S1 else S2

Numerical method	Jumping method
t1 = a < b t2 = c != d t3 = t1 && t2 if t3 != 0 goto Ltrue goto Lfalse Ltrue: ... code for S1 ... goto Lend Lfalse: ... code for S2 ... Lend:	if a < b goto L1 goto Lfalse L1: if c != d goto Ltrue goto Lfalse Ltrue: ... code for S1 ... goto Lend Lfalse: ... code for S2 ... Lend:

➢ Translation scheme(SDT) using a *numerical* representation for Booleans

Consider the grammar

E → E OR E | E AND E |NOT E | (E) | id relop id | TRUE | FALSE

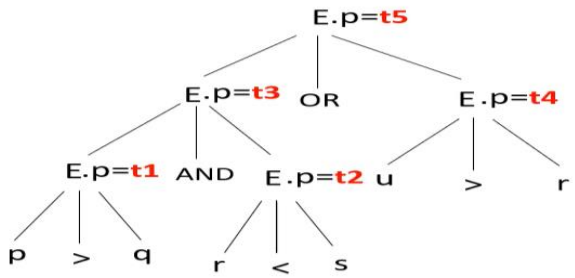
Production rule	Semantic actions
$E \rightarrow E_1$ **OR** E_2	{ E.place = newtemp(); emit(E.place ':=' E1.place 'OR' E2.place); }
$E \rightarrow E_1$ **AND** E_2	{ E.place = newtemp(); emit(E.place ':=' E1.place 'AND' E2.place); }
$E \rightarrow$ **NOT** E_1	{ E.place = newtemp(); emit(E.place ':=' 'NOT' E1.place); }
$E \rightarrow (E_1)$	{ E.place = E1.place ; }
$E \rightarrow$ **id**$_1$ *relop* **id**$_2$	{ E.place = newtemp(); emit('if' id1.place *relop*.op id2.place 'goto' nextstat + 3); emit(E.place ':=' '0'); emit('goto' nextstat + 2); emit(E.place ':=' '1'); }
$E \rightarrow$ **TRUE**	{ E.place = newtemp(); emit(E.place ':=' '1'); }
$E \rightarrow$ **FALSE**	{ E.place = newtemp(); emit(E.place ':=' '0'); }

emit(...) generate three-address code instruction.

E.place is a *synthesized attribute*.
It is computed from child nodes (E1, E2, id1, id2, etc.).
It represents the name of the temporary variable (like t1, t2) where the result of the expression is stored.

newtemp not an attribute, but a function generates a new temporary variable

nextstat External variable refers to the label or address of the next statement to be generated in the intermediate code.



Example:
p>q AND r<s OR u>r

Three Address Code:

```
100: if p>q goto 103
101: t1:=0
102: goto 104
103: t1:=1
104: if r<s goto 107
105: t2:=0
106: goto 108
107: t2:=1
108: t3=t1AND t2
109: if u>r goto 112
110: t4:=0
111: goto 113
112: t4:= 1
113: t5:= t3 OR t4
```

➢ **Translation scheme using Control flow (short-circuit) representation for Booleans**

A boolean expression is translated into three address instructions that evaluate the expression using create labels only when they are needed. Alternatively, unnecessary labels can be eliminated during the subsequent optimization phase. The semantic rules for different expressions are given in the diagram below.

emit(...) "generate a TAC instruction".

newlabel creates new label

E.*code* is the translation code for E, as usual it is a **synthesized** attribute.

The boolean expression *E* is associated with two labels (that are **inherited** attributes in the following semantic rules)

E → E_1 OR E_2	E_1.*true* := *E.true* E_1.*false* := *newlabel* E_2.*true* := *E.true* E_2.*false* := *E.false* *E.code* := E_1.*code* | | emit (E_1.*false* '**:**') | | E_2.*code*
E → E_1 AND E_2	E_1.*true* := *newlabel* E_1.*false* := *E.false* E_2.*true* := *E.true* E_2.*false* := *E.false* *E.code* := E_1.*code* | | emit (E_1.*true***:**') | | E_2.*code*
E → NOT E_1	E_1.*true* := *E.false* E_1.*false* := *E.true* *E.code* := E_1.*code*
E → (E_1)	E_1.*true* := *E.true* E_1.*false* := *E.false* *E.code* := E_1.*code*
E →id1 relop id2	*E.code* := emit ('**if**' id1 .*place* relop.op id2.*place* '**goto**' *E.true*) || *emit*('**goto**' *E.false*)
E → true	*E.code* := *emit*('**goto**' *E.true*)
E → false	*E.code* := *emit*('**goto**' *E.false*)

➢ **Flow-of-Control Statements**

Control flow statements in compiler design dictate the order in which instructions are executed within a program. Common control flow statements include if, else, while, and for, each enabling different ways to alter the sequential execution of code.

Grammar for the flow of control statements

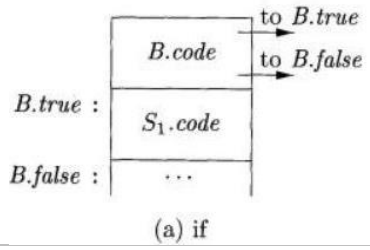
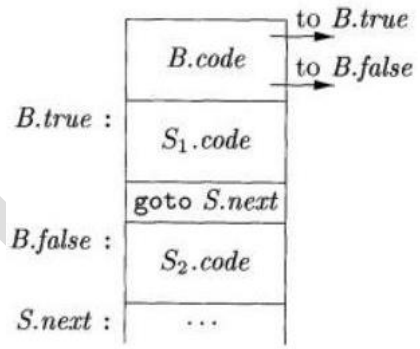
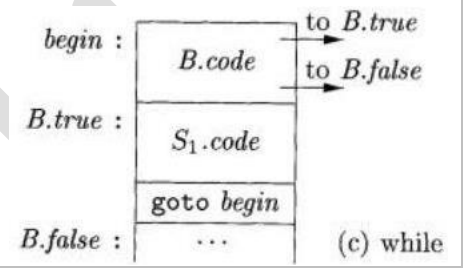
S → if(B) then S1

S → if(B) S1 else S2

S → while(B) S1

CD-I.H

Semantic rules for the flow of control statements. The semantic rules for some productions are given below

Production	Semantic Rule	
S → if B then S_1	B.true = newlabel(); B.false = S.next; S_1.next = S.next; S.code = B.code || gen(B.true, ":") || S_1.code	 (a) if
S → if B then S_1 else S_2	B.true = newlabel(); B.false = newlabel(); S_1.next = S.next; S_2.next = S.next; S.code = B.code || gen(B.true, ":") || S_1.code || gen("GOTO", S.next) || gen(B.false, ":") || S_2.code	 (b) if-else
S → while B do S_1	S.begin = newlabel(); B.true = newlabel(); B.false = S.next; S_1.next = S.begin; S.code = gen(S.begin, ":") || B.code || gen(B.true, ":") || S_1.code || gen("GOTO", S.begin)	 (c) while

- The boolean expression *E* is associated with two labels (that are **inherited** attributes in the following semantic rules)
  - *E.true* the label to which control flows if *E* is **true**, *E.false* the label to which control flows if *E* is **false**.
- In each of these productions, *S* is a flow of control statement associated with two attributes
  - *S.next* which is a label that is attached to the first 3-address statement to be executed after the code for *S* , *S.next* is an **inherited** attribute,
  - *S.code* is the translation code for *S*, as usual it is a **synthesized** attribute.
- Let's see figure (a) for 'if' code. B.code is the synthesized attribute for B. B.code can return two types of value- true and false. If it returns true then the control will jump to label B.true and S1.code is executed. If B.code returns false control will jump to label B.false the remaining part of the code.
- For 'if-else', see figure (b) above. If B.code is true then a new label B.true is created and statement S1.code is executed, for B.false S2.code is executed. And then the remaining code S.next will be executed.
- For 'while'(see figure c), if B.code is true then S1.code will be executed and control will again jump to label begin and the condition for B.code will be checked. It is done until B.code becomes false and then the remaining code will be executed(control will jump to label B.false).

➢ Backpatching

Backpatching is a technique used during intermediate code generation to handle forward jumps whose target addresses (labels) are not yet known at the time the jump instruction is generated.

Instead of leaving the jump target blank permanently, the compiler keeps a list of such incomplete jump instructions and fills in the correct target address later when it becomes known.

- Functions Used in Backpatching

Function	Purpose
`makelist(i)`	Creates a list containing instruction `i`
`merge(p1,p2)`	Combines two lists
`backpatch(p,i)`	Fills target address `i` into all instructions in list `p`

- Attributes Used

Attribute	Meaning
`B.truelist`	List of jumps when condition is true
`B.falselist`	List of jumps when condition is false
`S.nextlist`	List of jumps to the next statement

With Backpatching SDD for Boolean Expressions to generate TAC

A marker nonterminal M in the grammar causes a semantic action to pick up, at appropriate times, the index of the next instruction to be generated. The grammar is as follows:

B → B1 || MB2 | B1 && MB2 | !B1 | !(B1) | E1 rel E2 | true | false M → ϵ

***M* is a marker nonterminal that invokes a semantic action to pick up the index of the next instruction that is to be generated at appropriate times.**

The translation scheme for boolean expressions is as follows:

- Consider a semantic action (1) for the production B → B1 || MB2.
  If B1 is true, B is also true therefore *B1.truelist* become part of *B.truelist*.
  If B1 is false, we test B2, the target for the jumps *B1.falselist* must be the beginning of the code that is generated for B2
  We use the marker nonterminal *M* to get this target. This nonterminal produces as a synthesized attribute *M.instr*, the index of the next instruction before B2 is generated.

1) $B \rightarrow B_1 \ || \ M \ B_2$ { $backpatch(B_1.falselist, M.instr)$;
$B.truelist = merge(B_1.truelist, B_2.truelist)$;
$B.falselist = B_2.falselist$; }

2) $B \rightarrow B_1 \ \&\& \ M \ B_2$ { $backpatch(B_1.truelist, M.instr)$;
$B.truelist = B_2.truelist$;
$B.falselist = merge(B_1.falselist, B_2.falselist)$; }

3) $B \rightarrow \ ! \ B_1$ { $B.truelist = B_1.falselist$;
$B.falselist = B_1.truelist$; }

4) $B \rightarrow (\ B_1 \ )$ { $B.truelist = B_1.truelist$;
$B.falselist \ = \ B_1.falselist$; }

5) $B \rightarrow E_1 \ \mathbf{rel} \ E_2$ { $B.truelist = makelist(nextinstr)$;
$B.falselist = makelist(nextinstr + 1)$;
$gen('\texttt{if}' \ E_1.addr \ \mathbf{rel}.op \ E_2.addr \ '\texttt{goto} \ _')$;
$gen('\texttt{goto} \ _')$; }

6) $B \rightarrow \mathbf{true}$ { $B.truelist = makelist(nextinstr)$;
$gen('\texttt{goto} \ _')$; }

7) $B \rightarrow \mathbf{false}$ { $B.falselist = makelist(nextinstr)$;
$gen('\texttt{goto} \ _')$; }

8) $M \rightarrow \epsilon$ { $M.instr = nextinstr$; }

- We associate the semantic action *{M.instr = nextinstr;}* with the production *M* → *ϵ* to obtain the instruction index.
  nextinstr variable holds this index.
  Now, this value is backpatched onto B1.falselist when we see the remainder of the production B → B1 || M B2
- The semantic action (2) for B → B1 && M B2 is similar to (1).
  The action (3) for B → !B swaps the true and false lists and action (4) ignores parentheses.
  For simplicity action (5) generates two instructions, a conditional and an unconditional goto neither of which has its target filled, therefore they are placed on new lists pointed to by *B.truelist* and *B.falselist* respectively.

- Consider the expression **x < 100 || y > 200 && x != y**

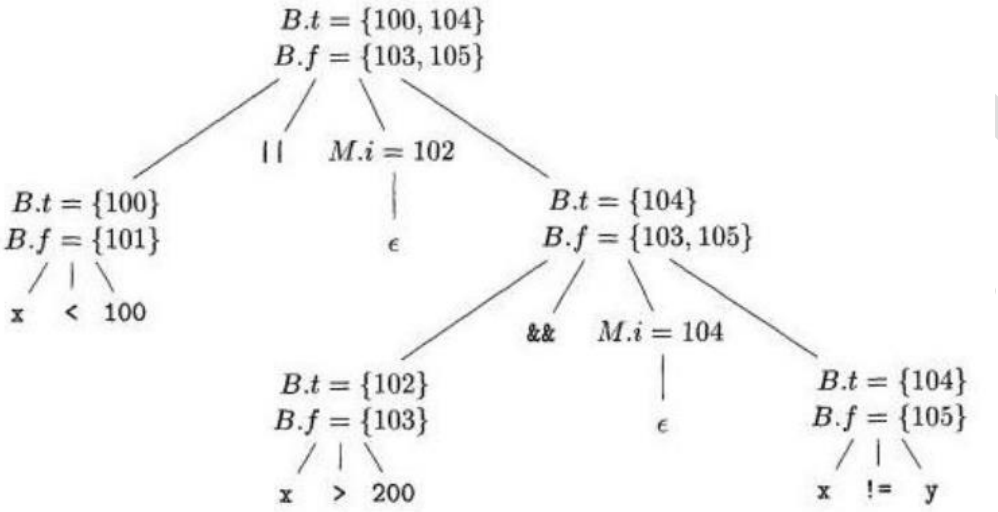
First pass where to go next is undefined

```
100: if x < 100 goto ____
101: goto ____
102: if y > 200 goto ____
103: goto ____
104: if x!=y goto ____
105: goto ____
106: true
107: false
```

Second pass using backpatching

```
100: if x < 100 goto  106
101: goto 102
102: if y > 200 goto  104
103: goto  107
104: if x!=y goto 106
105: goto 107
106: true
107: false
```

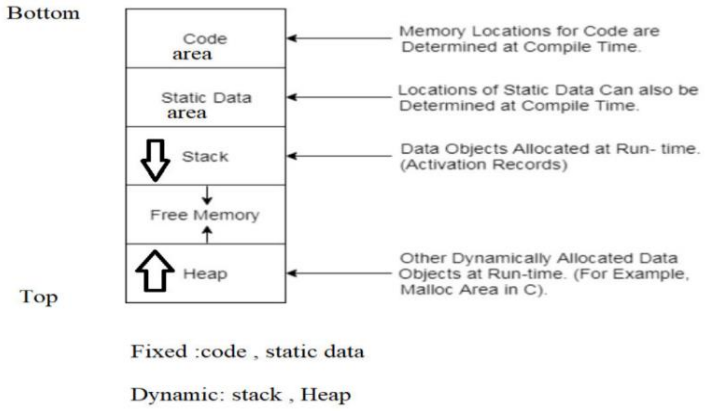
The image is an annotated parse tree for *x < 100 || x > 200 && x != y*.



UNIT – 4
RUN–TIME ENVIRONMENTS

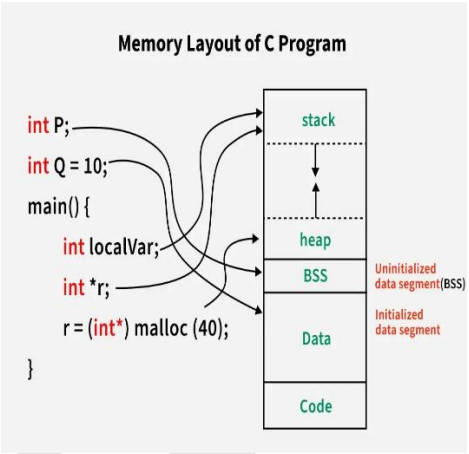
➢ Stack Allocation of Space

The run-time memory organization describes how memory is divided and used during program execution.
Typical subdivision of run-time memory



Fixed :code , static data

Dynamic: stack , Heap



1. Code Area
   - Stores the machine code/instructions generated by the compiler.
   - Contains instructions for functions/procedures.
   - Usually read-only.
   - Also called the text segment.
   - Exists throughout the execution of the program.
2. Static Area
   - Stores global variables and static variables.
   - Memory is allocated before program execution or during program initialization.
   - Variables remain allocated until program termination.
3. Stack
   - Used for function/procedure calls.
   - Each function call creates an activation record (stack frame).
   - Contains:
     Parameters , Local variables , Return address , Temporary values , Saved information
   - Activation record is removed when the function returns.
   - Generally follows LIFO (Last In, First Out).
4. Heap
   - Used for dynamic memory allocation.
   - Memory is allocated during program execution.
   - Objects/data can remain in the heap even after the function that created them terminates.
   - Memory is released using free() in C or by garbage collection in languages such as Java.

➢ Storage allocation strategies

1. Static Allocation
   - The size of data objects is known at compile time.
   - Memory is allocated before program execution.
   - The address of each variable is fixed during execution.
   - Variables remain allocated throughout the entire program execution.

- Used for: Global variables, Static variables, Constants

Allocation : Compile time Deallocation : Program termination

Limitation of static allocation

- The static allocation can be done only if the size of data object is known at compile time.
- The data structures cannot be created dynamically. In the sense that, the static allocation cannot manage the allocation of memory at run time.
- Recursive procedures are not supported by this type of allocation.

2. Stack Allocation

- In this the storage is organized as stack .This stack is also called control stack.
- As activation begins the activation records are pushed onto the stack and on completion of this activation the corresponding activation records can be popped.
- The locals are stored in the each activation record. Hence locals are bound to corresponding activation record on each fresh activation.
- The data structures can be created dynamically for stack allocation.
- Supports **recursion**.
- Allocation : During function call Deallocation : when function returns
- Limitation of stack allocation
  The memory addressing can be done using pointers an index registers. Hence this type of allocation is slower than static allocation.

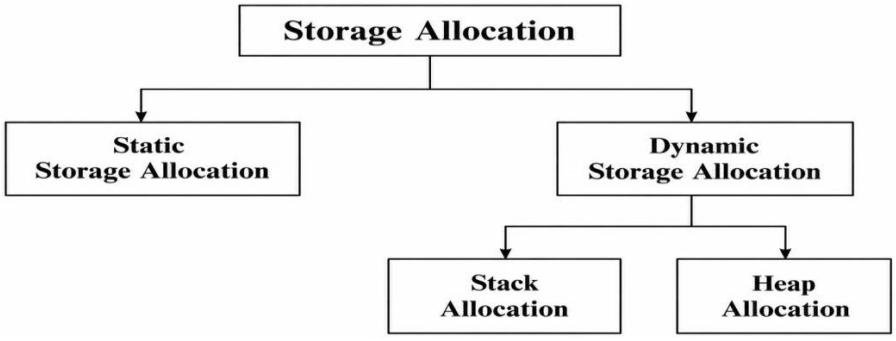
3. Heap Allocation

- Memory is allocated dynamically during program execution.
- Used when the size or lifetime of data cannot be determined at compile time.
- Data can have a lifetime independent of the function that created it.
- Used for: Dynamic arrays, Linked lists, Trees, Dynamically created objects

  Requires memory management such as:
  - malloc() / free() in C
  - new / delete in C++
  - Garbage collection in Java
- The efficient heap management can be done by Creating a linked list for the free blocks and when any memory is deallocated that block of memory is appended in the linked list.

The choice of storage allocation strategy depends mainly on the lifetime and size of the data. Static allocation is used for data with fixed lifetime, stack allocation for procedure activations, and heap allocation for dynamically created data.

➢ Static Versus Dynamic Storage Allocation



S.No	Static Memory Allocation	Dynamic Memory Allocation
1	When the allocation of memory performs at the compile time, then it is known as static memory.	When the memory allocation is done at the execution or run time, then it is called dynamic memory allocation.
2	The memory is allocated at the compile time.	The memory is allocated at the runtime.
3	In static memory allocation, while executing a program, the memory cannot be changed.	In dynamic memory allocation, while executing a program, the memory can be changed.
4	Static memory allocation is preferred in an array.	Dynamic memory allocation is preferred in the linked list.
5	It saves running time as it is fast.	It is slower than static memory allocation.
6	Static memory allocation allots memory from the data segments.	Dynamic memory allocation allots memory from the heap.
7	Static memory allocation is less efficient as compared to Dynamic memory allocation.	Dynamic memory allocation is more efficient as compared to the Static memory allocation.
8	This memory allocation is simple.	This memory allocation is complicated.
9	Does not have memory fragmentation issue	Can lead to memory fragmentation
10	Handled by the compiler	Handled by the programmer
11	Wastage of memory	Nominal wastage of memory

➢ Activation tree

An *activation* is the execution of a procedure. A procedure has a beginning and an ending delimiter and everything in between is the body of the procedure.

An *activation record* stores all information that is required to call a procedure.

An activation tree is a graphical representation of the control flow of a program, specifically showing how procedures or functions are called and executed. It's a tree-like structure where each node represents an activation of a procedure, and the edges depict the calling relationships between them.

1. The sequence of procedure calls corresponds to a **preorder** (Root-left-right) traversal of the activation tree.

2. The sequence of returns corresponds to a **postorder** (left-right- Root) traversal of the activation tree

Activation Tree Structure:

- Each node in the tree represents a single activation of a procedure.
- The root of the tree is the activation of the program's main function.
- If procedure A calls procedure B, the node for B's activation will be a child of the node for A's activation in the tree.
- The depth of a node in the tree corresponds to the nesting level of procedure calls.

Example :

```
    void C() {
    ------
    -----
    }

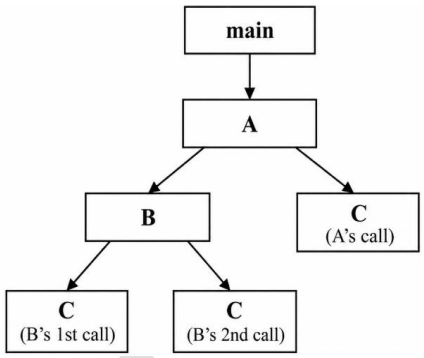
    void B() {
       C();
       C();
    }

    void A() {
       B();
       C();
    }

int main() {
   A();
}
```

```
System starts main
    enter main
        enter A
            enter B
                enter C
                exit C
                enter C
                exit C
            exit B
            enter C
            exit C
        exit A
    exit main
main ends
```

Activation Tree



Example:

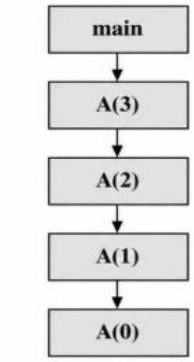
```
void A(int n) {
    if (n > 0)
        A(n-1);
}

int main() {
    A(3);
}
```

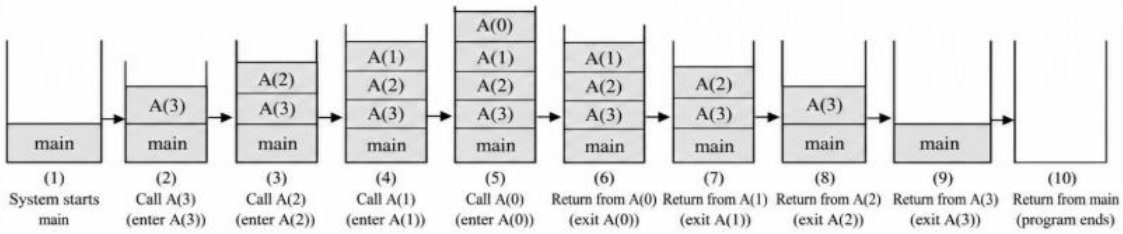
Sequence of calls and returns

```
System starts main
     enter main
         enter A(3)
            enter A(2)
               enter A(1)
                  enter A(0)
                  exit  A(0)
               exit  A(1)
            exit  A(2)
         exit  A(3)
      exit main
main ends
```

Activation Tree

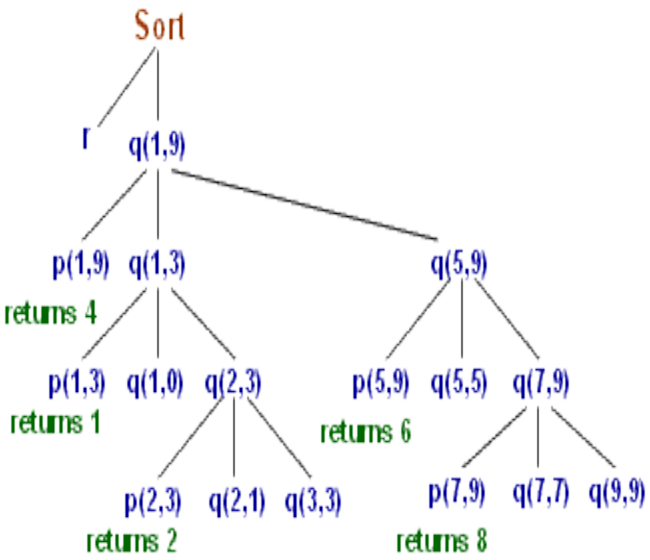


Stack Allocation During Execution



- The order of activations (procedure calls) corresponds to a preorder traversal of the call tree.
- The order of de-activations (procedure returns) corresponds to postorder traversal of the call tree.
- If execution is currently in an activation corresponding to a node N of the activation tree, then the activations that are currently live are those corresponding to N and its ancestors in the tree. These live activations were called in the order given by the root-to-N path in the tree, and the returns will occur in the reverse order.

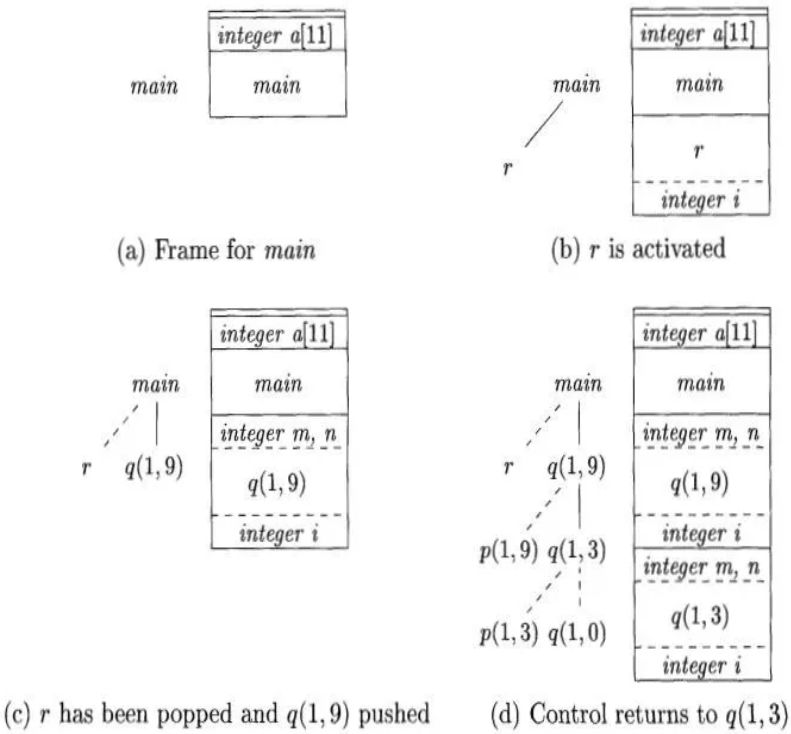
Example:




```
main() {
    Int n;
    readarray();
    quicksort(1,n);
}
quicksort(int m, int n) {
   Int i= partition(m,n);
    quicksort(m,i-1);
    quicksort(i+1,n);
}
```

The flow of control of the call quicksort(1,9) would be like

Execution begins ..
enter readarray
exit readarray
enter quicksort(1,9)
enter partition(1,9)
exit partition(1,9)
enter quicksort(1,3)
……
……
exit quicksort(1,3)
enter quicksort(5,9)
……
……
exit quicksort(5,9)
exit quicksort(1,9)
Execution terminates



➢ Activation Records:

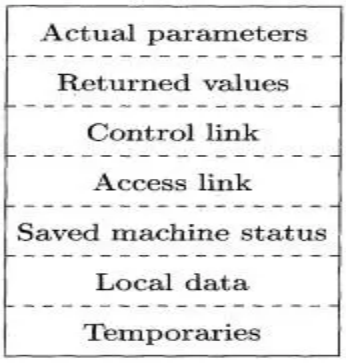
When a function is called, an activation record (also known as a stack frame) is created and pushed onto the stack.

When the function returns to its caller, the activation record for this function call is popped off the stack and those local variables are no longer known to the program.

This record contains information about the function, including:

The components of an activation record, as shown in the image, are:

- Return value:
  Stores the value returned by the function or procedure upon completion.
- Actual parameters:
  Holds the values of the arguments passed to the function or procedure when it is called.
- Machine status:
  Information about the state of the machine's registers and program counter before the function call, allowing the program to resume execution correctly upon return.



- Control links / Dynamic link :
  The control link points to the activation record of the caller (the procedure/function that called the current function).
  The control link points to the activation record of the caller (the procedure/function that called the current function).
  Example:
  If main() calls f(), and f() calls g(), then:
  g's AR has a control link pointing to f's AR.
  f's AR has a control link pointing to main's AR.
- Access links / static link :
  Used to access non-local variables in languages that support nested procedures.
  Represents the static/lexical relationship, not the calling relationship.
  Control Link = follows **dynamic call chain** (execution order).
  Access Link = follows **static nesting chain** (scope structure).
- Local data:
  Space allocated for the local variables declared within the function or procedure.
- Temporaries:
  Storage for intermediate values generated during expression evaluation within the procedure.

The "Direction of growth" arrow indicates that new activation records are pushed onto the stack as procedures are called, growing downwards in memory, and popped off when they terminate.

Example :

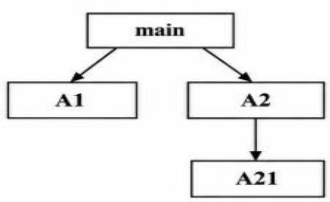
Program Code

```
void main()
{
    A1();
    void A1()
    {
        A2();
    }
    void A2()
    {
        void A21()
        {
            A1();
        }
        A21();
    }
    A1();
}
```

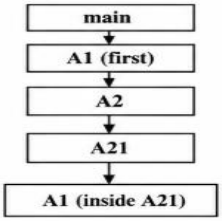
Function Call, Control Link and Access Link

Function Call	Control Link (caller)	Access Link (lexical parent)
main	–	–
A1 (first call from main)	main	main
A2 (called from A1)	A1	main
A21 (called from A2)	A2	A2
A1 (inside A21)	A21	main

Lexical (Nesting) Structure



Call Sequence (During Execution)



Calling Sequences

Calling sequences consists of code that allocates an activation record on the stack and enters information in its fields. They are used to implement procedure calls.

A *return sequence* restores the state of the machine so that a calling sequence can continue its execution after the call.

Code in a calling sequence is divided between the *caller(calling procedure)* and the *Callee (procedure being called)*.

Calling sequence actions

Caller**:**

- Evaluates the actual parameters of the procedure call.
- Stores the parameters in the called procedure's activation record.
- Saves the return address (the next instruction to execute after the call).
- Saves the caller's current machine state (e.g., register values).
- Updates the stack pointer (SP) to point to the new activation record.
- Transfers control to the called procedure by jumping to its first instruction.

Callee**:**

- Allocates and initializes space for its local variables.
- Begins execution of its code.

Return sequence actions

Callee**:**

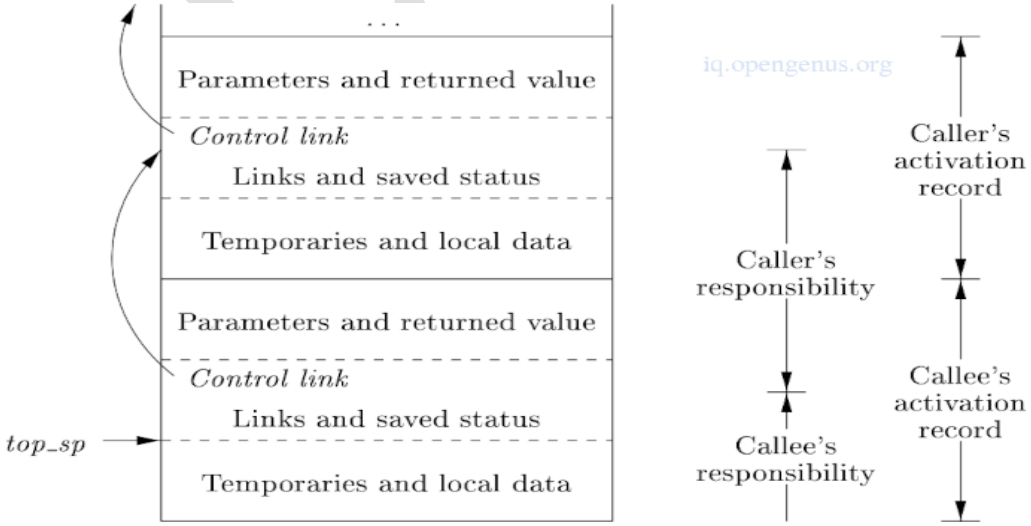
- Stores the return value (if any).
- Restores the caller's saved registers and machine state.

Caller**:**

- Retrieves the return value.
- Deallocates the called procedure's activation record by moving the stack pointer back to its original position.
- Resumes execution from the return address.

That is, if a procedure is called from *n* points then the portion of the calling sequence that is assigned to the caller is generated *n* times while the portion for the callee sequence is only generated once.

The division of tasks between caller and callee



The register ***top_sp*** points to the end of the machine-status field in the current top AR and since this position is known to the caller, the caller is responsible for setting *top_sp(Top of the stack pointer)* before control is passed to the callee.

Variable-Length Data on the Stack

Variable-length data is data whose size is determined at run time

There are two flavours of variable-length data.

- Data obtained by malloc/new are stored in the heap instead of the stack.
- Data, such as arrays with bounds determined by the parameters are stored in stack

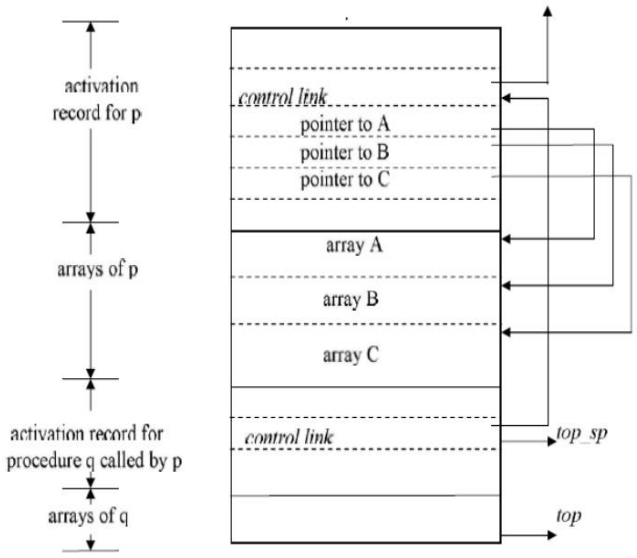
```
void P(int n)
{
   int A[n];
}
```

Since the size of A is not known at compile time, it cannot be stored as a fixed field in the activation record. Instead:

- The actual array is dynamically allocated on the stack.
- A pointer to the beginning of the array is stored in the fixed part of the activation record.
- The pointer's position in the activation record is known at compile time.
- The compiler-generated code uses this pointer to access the array elements.
- The variable-length data has a stack-like lifetime—it exists while its procedure is active.
- If P calls Q, P's variable-length data remains allocated while Q executes.
- When Q returns, its activation record is removed; when P returns, P's variable-length data is released.

In Fig Procedure *p* has three local arrays whose sizes cannot be determined at compile time.

Storage for these arrays is not part of *p's* AR even though it appears on the stack. Only a pointer to the beginning of each array appears in the AR and therefore during *p's* execution these pointers are known offsets from the top-of-stack pointer and the target code is able to access array elements through these pointers(*top* and *top_sp*).



Access to dynamically allocated arrays

top and top_sp

- top_sp → used to access the fixed fields of the current activation record.
- top → indicates the current top/end of the allocated stack area.

As variable-length data is allocated or deallocated, `top` changes accordingly.

➢ Access to Nonlocal Data on the Stack

NON-LOCAL DATA

A function may sometimes need to access data outside its scope. Such data is referred to as *non-local data* on the stack.

In compiler design, nonlocal data means a variable that is not declared in the current procedure, but is declared in an outer/nested procedure. Access to non-local data depends on the scope structure, not the call stack order.

- Examples:
  - Global variables (accessible in all functions).
  - Variables from outer functions (in languages supporting nested functions, like Python or Pascal) or outer block

Case 1: Global Variables

Global variables are **stored in a fixed area of memory**, usually the **data segment**.

- Any function can access them directly using their **address**.

```
int x = 100; // global
void func() {
   printf("%d\n", x); // directly accessed
}
```

- Here, the compiler generates code that **directly refers to the memory location of x**.

Case 2 : Nested Procedures

A nested procedure is a procedure or function that is defined inside another procedure or function.

The outer procedure is called the enclosing procedure, and the inner procedure is called the nested procedure.

A nested procedure can access variables declared in its enclosing procedure.

```
procedure A()
{
   int x;

   procedure B()
   {
      int y;
      print(x);
   }

   B();
}
```

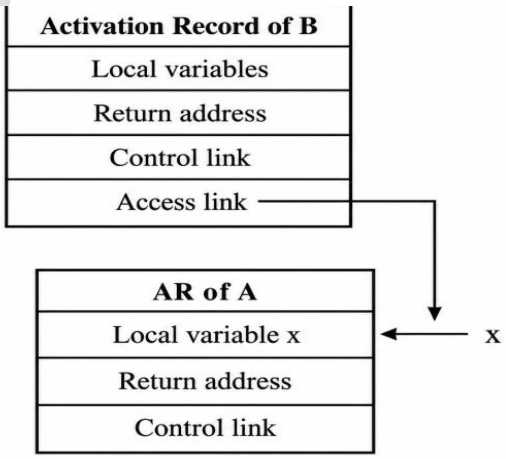
In this,

A() → Enclosing procedure

│

└── B() → Nested procedure

`x` is local to A(), but it is nonlocal to B().
Therefore, when `B()` executes, it needs a way to find `x`.
This is done using an **access link (static link)**.



Nesting depth Nesting depth is the number of levels of procedure nesting in a program. It tells us how deeply a procedure is nested inside other procedures. Nesting depth is important when accessing nonlocal variables.

A() depth 1 | └── B() depth 2 | └── C() depth 3 If: current procedure has nesting depth d variable is declared at nesting depth k then the number of access-link traversals is: **d - k**	If `C()` wants to access a variable declared in `A()`: C's AR | | Access link ↓ B's AR | | Access link ↓ A's AR | ↓ X `C()` must follow **2 access links** to reach `A()`.

Accessing Non-Local Data

Two strategies for accessing non-local data on the stack in Compiler Design are –

1. Access Links 2. Displays.

1. Access Link

Access Links in one activation record are links that are used to refer to the non-local data in other activation records and provide access to it. A chain of access links is formed on the top of the stack and represents the program's scope or static structure.

The no. of hops to be done from one AR to other AR to get a variable declared is

Hops are the number of activation records (ARs) or static links that must be followed to establish the access link of the called procedure. Let

- dcaller = nesting depth of caller
- dcallee = nesting depth of callee

Condition	Number of static-link hops
$d_{caller} < d_{callee}$	$d_{callee} - d_{caller}$
$d_{caller} \geq d_{callee}$	$d_{caller} - d_{callee} + 1$

Example :

A depth 1 | B depth 2 | C depth 3 Suppose a procedure at depth 3 calls a procedure at depth 2: Caller = C → depth 3 Callee = B → depth 2 Then: $3-2+1=2$ So the compiler follows 2 static/Access links to determine the appropriate access link.	Suppose a procedure at depth 1 calls a procedure at depth 2: Caller = A → depth 1 Callee = B → depth 2 Then: 2 - 1 =1 So the compiler follows 1 static/Access links to determine the appropriate access link.

2. Displays

One problem with the access-link approach to nonlocal data is that if the nesting depth gets large, we may have to follow long chains of links to reach the data we need.

A display is a data structure used to access non-local variables efficiently when procedures are nested.

Each procedure has an activation record (AR) on the stack.

A display is an array of pointers where each entry points to the activation record at a particular nesting depth.

Suppose C() wants to access variable x declared in A(): Since A() has nesting depth 1, the compiler uses D[1]. So the variable can be accessed directly.

With access links, we would need: C → B → A → x

But with a display: C → D[1] → A → x

Therefore, displays provide direct access to the required activation record.

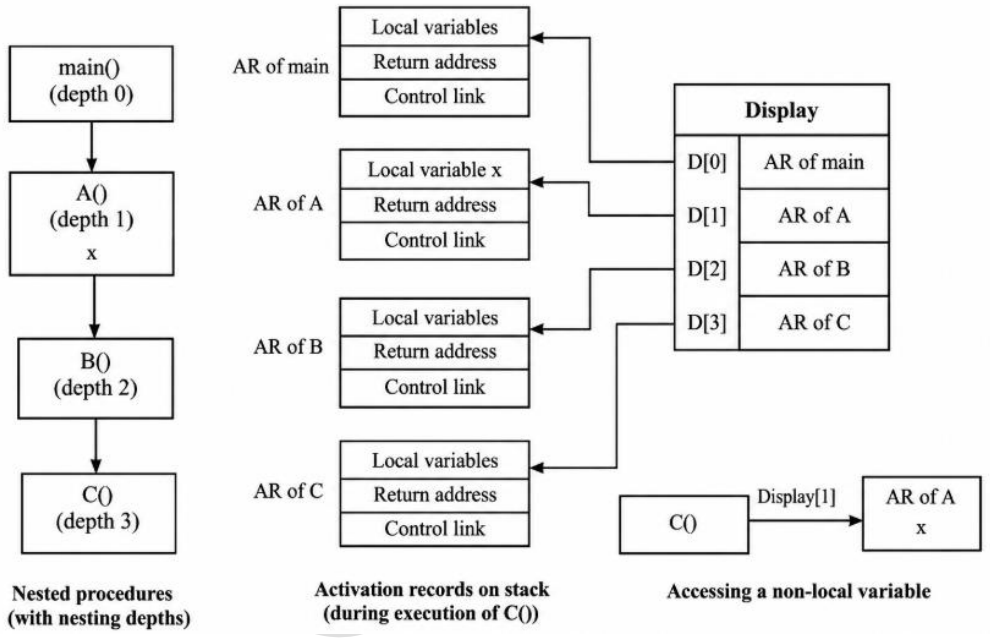
If C() wants a variable from:

main → use D[0]

A → use D[1]

B → use D[2]

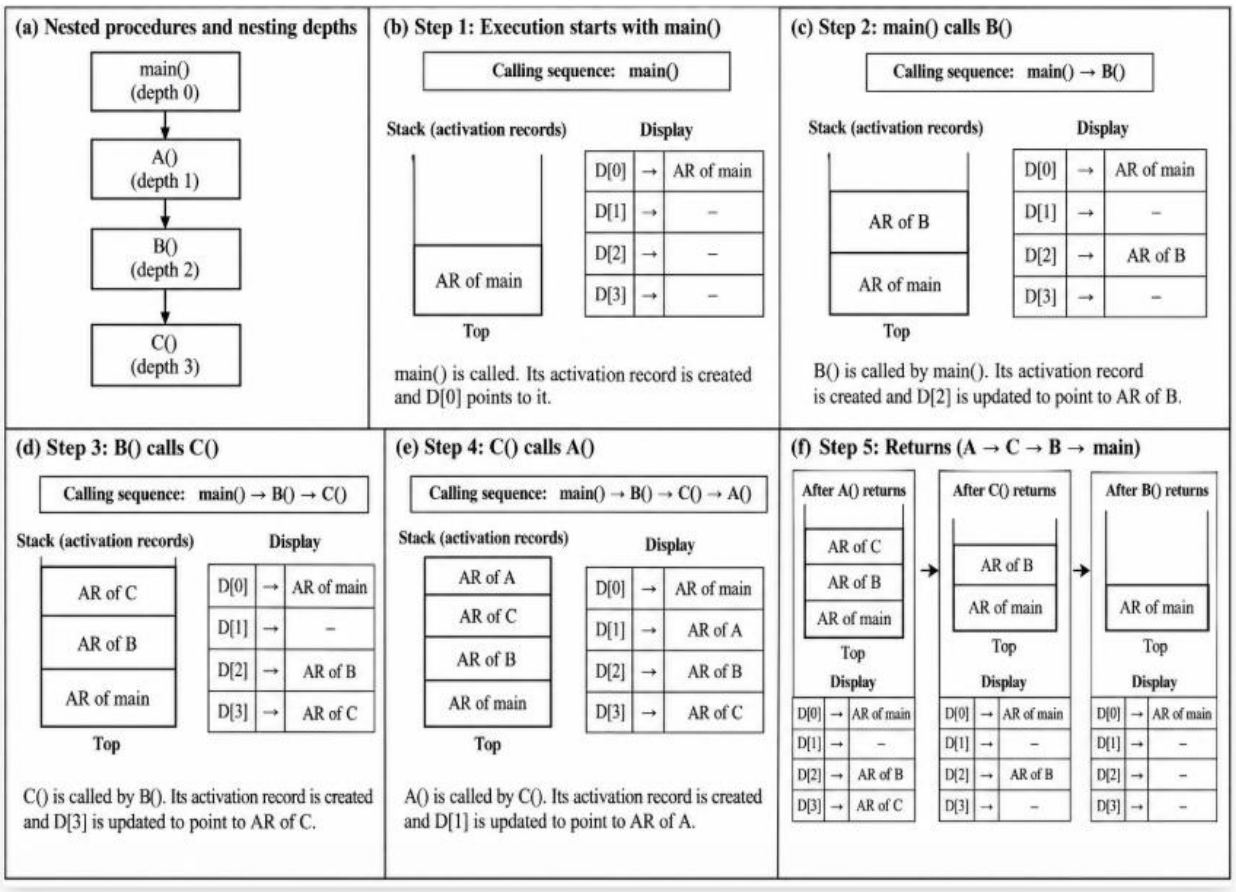
C → use D[3]



The display "displays" the currently active activation record for each nesting level.

Display during procedure calls

Suppose calling sequence is main → B→ C → A then



➢ Heap management

Heap is a memory area used for dynamically allocated data.

Memory can be allocated and deallocated during program execution.

It is useful when the size or lifetime of data is not known at compile time.

Examples: linked lists, trees, graphs, dynamic arrays.

Dynamically allocated objects are stored in the heap, but pointers to those objects are not necessarily stored in the heap. A pointer may be on the stack, in global memory, or in a register.

Importance of Heap Management in Compiler Design

1. Dynamic Memory Allocation
   - Allocates memory at run time according to program requirements.
   - Supports dynamic data structures such as:
   - Linked lists, Trees, Graphs, Dynamic arrays
2. Memory Reusability
   - Memory that is no longer required can be deallocated and reused.
   - Helps improve memory utilization.
   - Proper management helps reduce fragmentation.
3. Support for Dynamic Languages
   - Languages such as Python and JavaScript make extensive use of dynamically allocated objects.
   - Efficient heap management is important for their performance.
4. Flexible Storage
   - Unlike stack allocation, heap memory is not restricted to LIFO order.
   - Objects can have lifetimes independent of a particular procedure call.

The Memory Manager in Heap Storage

The memory manager for heap storage is responsible for dynamically allocating and deallocating memory during program execution. Main Functions are

1. Allocation

When a program requests memory, the memory manager:

Searches for a suitable free block.

Allocates the required memory.

Returns its address to the program.

If sufficient space is unavailable, more memory may be obtained from the operating system.

2. Deallocation

When memory is no longer needed, it is released.

The memory manager marks the block as free.

The free space can then be reused for future allocations.

➢ Techniques for Heap Management

There are three commonly used techniques:

1 Explicit Heap Management:

This technique requires the programmer to manually allocate and deallocate memory using specific functions or keywords provided by the programming language.

Examples include the malloc() and free() functions in C/C++.

Explicit heap management gives the programmer full control over memory allocation and deallocation

It requires careful tracking of memory usage to avoid memory leaks or accessing deallocated memory.

Example : C Langauage

```
int *p = malloc(10 * sizeof(int));
free(p);
```

2 Implicit Heap Management:

Implicit heap management is performed automatically by the compiler or runtime environment without explicit instructions from the programmer.

Garbage collection is a common example of implicit heap management, where the runtime system periodically identifies and deallocates memory that is no longer reachable.

This relieves the programmer from manually managing memory but may introduce some overhead.

Examples: Java, Python, JavaScript.

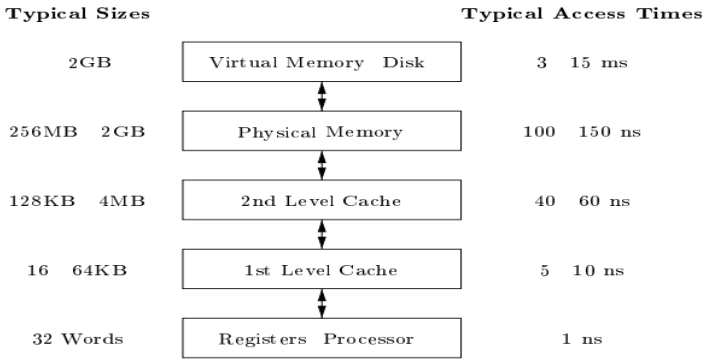
3 Hybrid Heap Management:

Hybrid heap management combines elements of explicit and implicit techniques. It involves a combination of manual memory management and garbage collection. The programmer can manually allocate and deallocate memory, but the garbage collector handles memory reclamation for objects that are no longer reachable. This approach provides flexibility and convenience while maintaining memory efficiency.

Example : C++

➢ Memory Hierarchy of a Computer

The memory hierarchy plays a crucial role in optimizing code execution. It refers to the organization of memory components in a computer system based on their access speed and cost, with faster, smaller, and more expensive components closer to the CPU and slower, larger, and cheaper components further away.



- Registers:

These are the fastest memory, located within the processor, with access times around 1 ns and sizes typically around 32 words.

- Cache Memory (1st & 2nd Level):

These levels provide faster access than main memory, with 1st Level Cache being faster and smaller than 2nd Level Cache. Access times range from 5-10 ns for 1st Level and 40-60 ns for 2nd Level.

- Physical Memory (RAM):

This is the main memory, significantly larger and slower than cache, with typical sizes from 256MB to 2GB and access times around 100-150 ns.

- Virtual Memory/Disk:

This is the slowest and largest level, using disk space to extend the apparent size of physical memory, with access times in the milliseconds range (3-15 ms).

➢ Heap Memory Allocation Strategies:

1. **Best-Fit:**

This strategy allocates the requested memory block in the smallest available hole that is large enough. While it aims to preserve larger holes for future large requests, it can lead to more frequent searches and potentially create many small, unusable fragments.

2. **Worst-fit** :

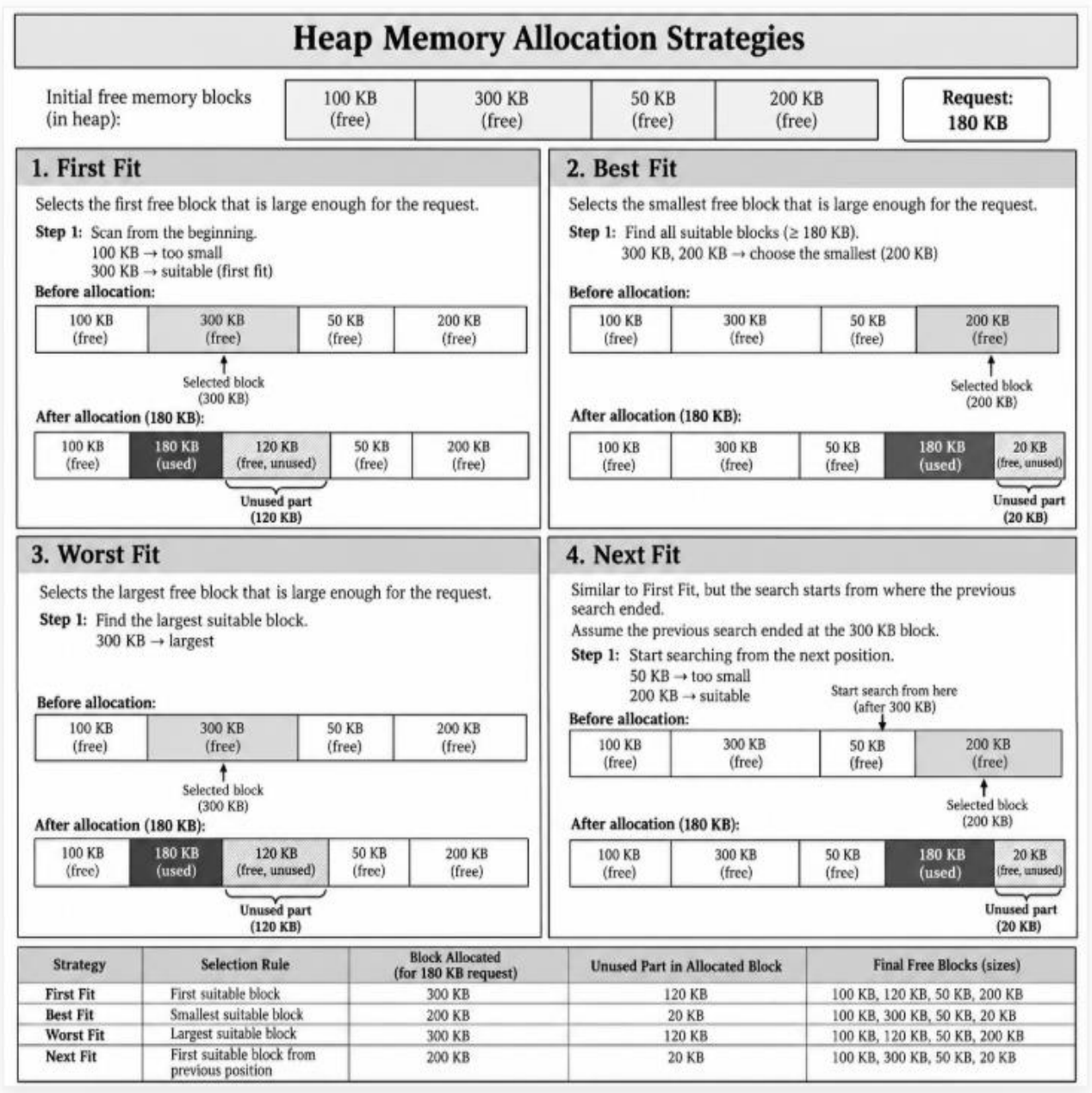
This strategy allocates the requested memory block in the largest available hole that can fit

3. **First-Fit:**

This strategy allocates the requested memory block in the first available hole encountered that is large enough. It is generally faster than best-fit but can lead to external fragmentation by consuming larger holes early on.

4. **Next-Fit:**

Similar to first-fit, but it starts searching from where the last allocation left off, potentially distributing allocations more evenly and reducing the likelihood of repeatedly splitting the same large hole.

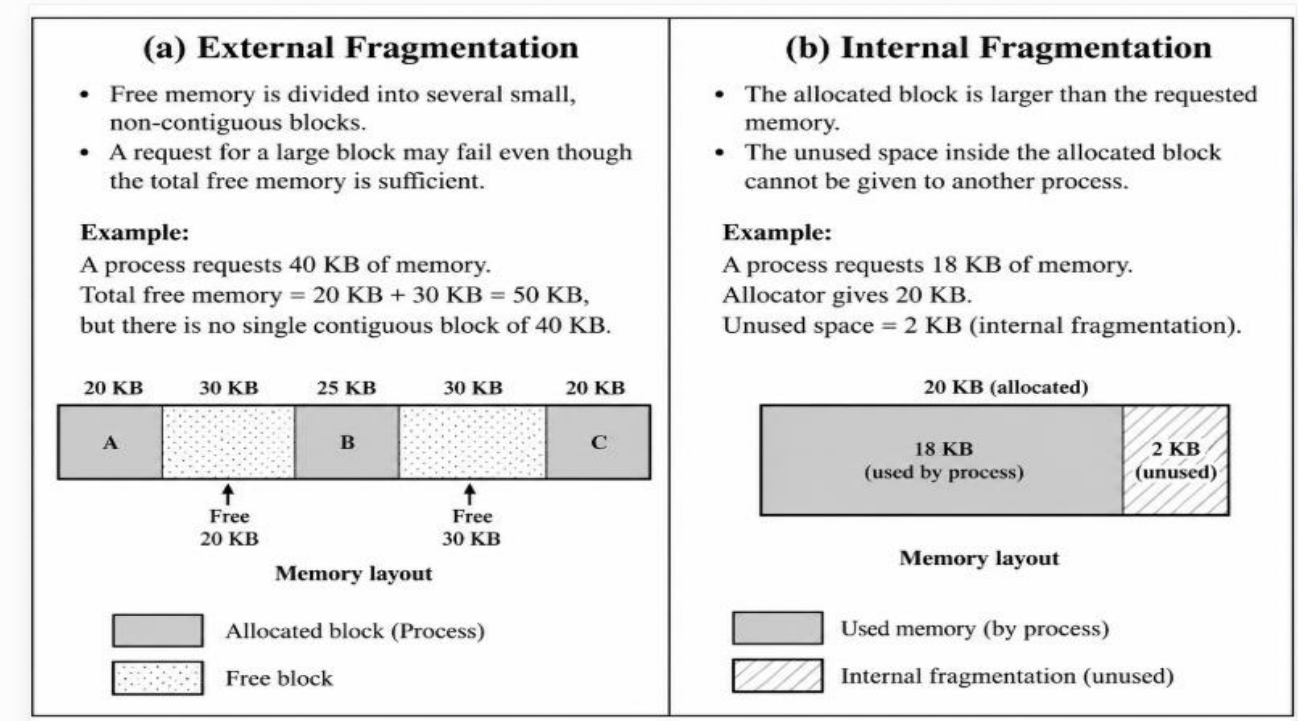


Strategy	Selection Rule	Block Allocated (for 180 KB request)	Unused Part in Allocated Block	Final Free Blocks (sizes)
First Fit	First suitable block	300 KB	120 KB	100 KB, 120 KB, 50 KB, 200 KB
Best Fit	Smallest suitable block	200 KB	20 KB	100 KB, 300 KB, 50 KB, 20 KB
Worst Fit	Largest suitable block	300 KB	120 KB	100 KB, 120 KB, 50 KB, 200 KB
Next Fit	First suitable block from previous position	200 KB	20 KB	100 KB, 300 KB, 50 KB, 20 KB

➢ Problems in Heap Memory Management

Fragmentation

- Fragmentation is **created over time** as programs **allocate** and **free** memory blocks of **different sizes and lifetimes**.
- At the beginning of program execution, the heap is one contiguous unit of free space. As the program allocates and deallocates memory, this space is broken up into free and used chunks of memory.
- Repeated allocation and deallocation can leave unused gaps in the heap.
- There are two main types:
  1. External fragmentation 2 Internal fragmentation



Reducing Fragmentation

Fragmentation is an unwanted problem where the memory blocks cannot be allocated to the processes due to their small size and the blocks remain unused. Several techniques are employed to achieve this

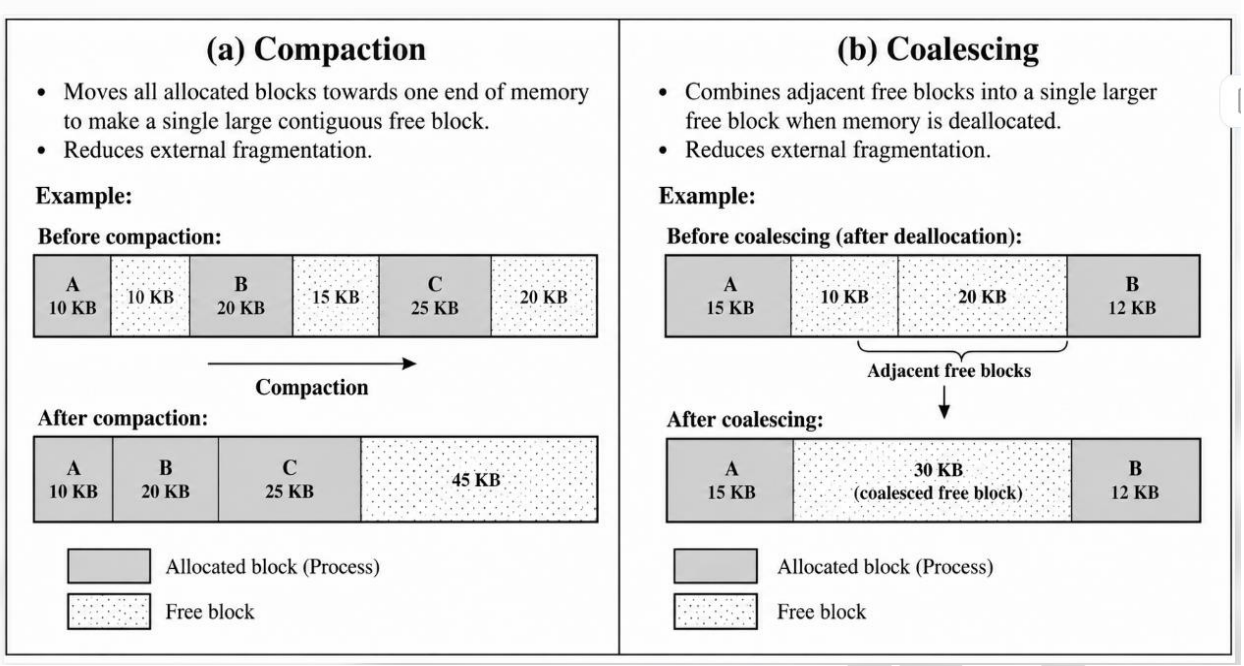
1. Using Compaction 2. Using Coalescing

i) Compaction

This involves moving allocated memory blocks to consolidate free space into larger, contiguous blocks. This is often performed during garbage collection in languages with automatic memory management.

ii) Coalescing

- Coalescing is the process of merging adjacent free memory blocks into one larger free block after deallocation. It helps reduce external fragmentation.
- When a memory block is deallocated, it becomes free.
- If the blocks next to it are also free, they are merged (coalesced) into one larger block.
- This creates a larger contiguous space for future allocations.
- Adjacent blocks must be free to coalesce.
- If the memory is divided into fixed-size chunks, coalescing is generally not required because all chunks have the same size.
    - A bitmap can be used for fixed-size chunks:- 1 → chunk is occupied 0 → chunk is free
    - On deallocation, change 1 → 0.
    - For allocation, find a 0 and change it to 1.

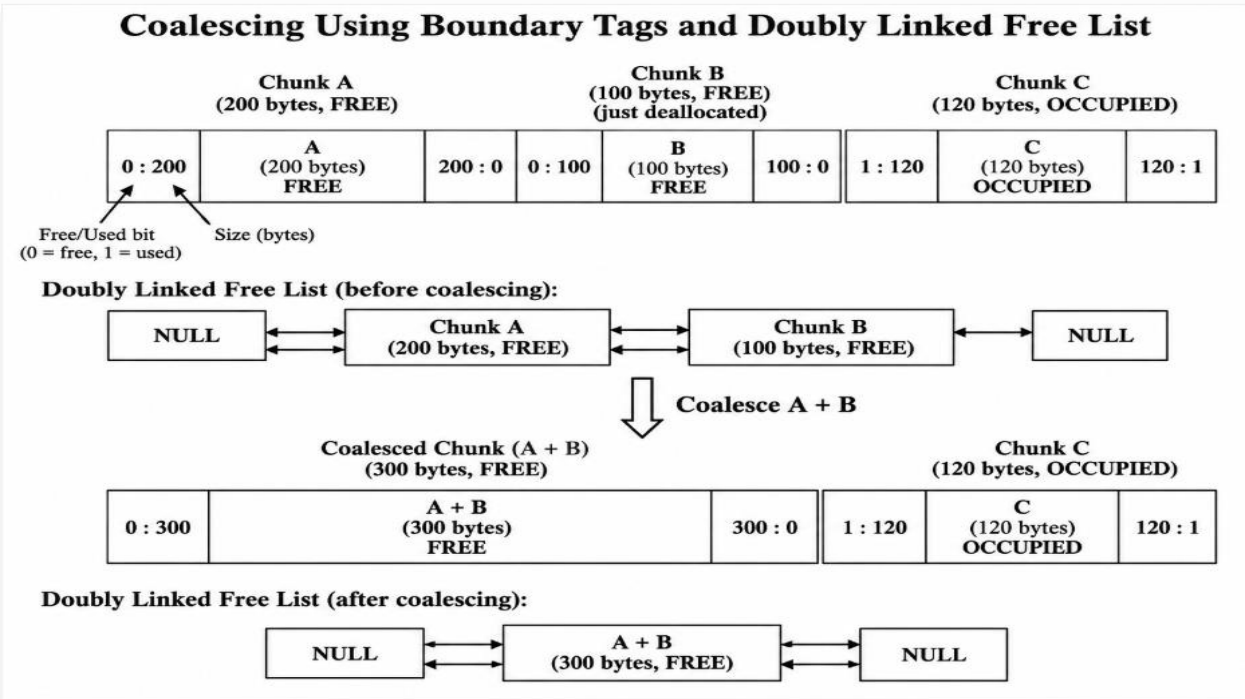


Techniques Used for Coalescing

1. Boundary Tags
    - Information is stored at the beginning and end of each memory block.
    - It indicates:
        - Whether the block is free or occupied.
        - The size of the block.
    - This allows the memory manager to quickly check whether neighboring blocks are free.
2. Doubly Linked Free List
    - All free blocks are connected using a doubly linked list.
    - Each free block contains pointers to the previous and next free blocks.

- When blocks are coalesced, the corresponding entries are removed/updated in the free list

.



➢ Compaction VS Coalescing

Compaction	Coalescing
Moving allocated blocks together to combine all scattered free space into one large continuous region.	Merging adjacent free blocks into a single larger free block.
Physical operation — actually moves data in memory.	Logical operation — updates metadata (no data movement).
Before | Alloc(100) | Free(50) | Free(100) | Alloc(200) | Free(150) | After compaction | Alloc(100) | Alloc(200) | Free(300) |	Before | Alloc(100) | Free(50) | Free(100) | Alloc(200) | Free(150) | After coalescing | Alloc(100) | Free(150) | Alloc(200) | Free(150) |
Eliminate external fragmentation globally.	Reduce external fragmentation locally.
Moves allocated data	Doesn't move allocated data
It is performed Periodically by garbage collector or OS when fragmentation is high	It is performed when When a block is freed (free() or delete)
Used in Automatic systems (Java, Python, JVM, OS memory managers)	Used in Manual allocators (C/C++, heap managers)
Completely removes fragmentation	Reduces fragmentation partially

➢ Locality in Programs

- Locality of reference or "locality in programs" in computer architecture is a concept that explains how programs access memory. It is also known as the Principle of Locality.
- Locality of reference refers to the tendency of a program to access the same or nearby memory locations repeatedly.
- A heap manager can try to improve locality by allocating related objects close together. Because Heap management decides where dynamically allocated objects are placed in memory.
- Repeated allocation and deallocation can produce external fragmentation:
- There are two primary types of locality:
  1. Temporal locality: Recently accessed data is likely to be accessed again soon.
  2. Spatial locality: Data located near a recently accessed memory location is likely to be accessed soon.

1. Temporal Locality:

This refers to the tendency of a program to repeatedly access the same memory location or instruction within a short time frame. Loops and frequently used variables are common examples that exhibit strong temporal locality.

Example:
```
for (i = 0; i < 1000; i++)
    sum += A[i]
```

The ‘sum’ is accessed repeatedly. The same variable is read and updated repeatedly.
Therefore, sum is an example of temporal locality.

2. Spatial Locality:

If a program accesses one memory location, it is likely to access nearby memory locations soon.
Example:
Array traversals and sequential instruction fetching are prime examples of spatial locality.
Example :

```
int A[5] = {10, 20, 30, 40, 50};
for (int i = 0; i < 5; i++)
   printf("%d", A[i]);
```

The program accesses: `A[0] → A[1] → A[2] → A[3] → A[4]`
These array elements are stored in contiguous/nearby memory locations. Therefore, this is an example of spatial locality.
Finally, Temporal = Same memory again Spatial = Nearby memory next

➢ Heap Deallocation

It can be done in 2 ways a) manual deallocation b)Automativ deallocation
a) Manual Deallocation

- A manual deallocation request is when the programmer explicitly releases heap memory that is no longer needed.
- The programmer decides when the memory is no longer required.
- The programmer must explicitly call a deallocation function to release it.
- In C, `free()` is used to release memory allocated using `malloc()`
- In C++, `delete` is used to release memory allocated using `new`.
- Incorrect manual deallocation can cause:
  - Memory leaks – allocated memory is not released.
  - Dangling pointers – a pointer refers to memory that has already been released.
  - Double deallocation – the same memory is released twice.

Memory Leak

A memory leak occurs when allocated memory is not released, and the program loses the reference/pointer to that memory.

```
void fun()
{
   int *p = malloc(100 * sizeof(int));
   // use memory     forgot to call free(p)
}
```

Dangling Pointer

A dangling pointer occurs when a pointer still refers to a memory location after that memory has been deallocated.

```
int *p = malloc(100 * sizeof(int));
free(p);      // memory released
*p = 10;      // WRONG: accessing freed memory
```

B) Automatic Deallocation

- In automatic deallocation, the runtime system automatically releases heap memory when the allocated object is no longer needed.
- The programmer does not explicitly call a deallocation function.
- This is commonly achieved using garbage collection.
- Example in Java:
    - Student s = new Student();
    - s = null;
- Once the Student object is no longer reachable, the garbage collector can identify it and reclaim its memory.
- Advantages
  - Reduces programmer responsibility.
  - Helps prevent memory leaks and dangling-pointer errors.

➢ Garbage Collection

Garbage collection refers to the automatic process of identifying and reclaiming memory that is no longer in use by a running program

Performance Metrics

1. Overall Execution Time
   - Measures the total time taken by the program, including garbage collection.
   - A good GC should add as little extra time as possible.
2. Space Usage
   - Measures how efficiently the GC uses available memory.
   - The GC should reduce fragmentation and use memory efficiently.
3. Pause Time
   - The time for which the application is stopped while GC is running.
   - Lower pause time is especially important for interactive and real-time applications.
4. Program Locality
   - Refers to how closely related data is placed in memory.
   - A good GC can improve:
     - Temporal locality: Frequently used data can be reused efficiently.
     - Spatial locality: Data used together can be placed close together.

A good garbage collector should minimize execution time and pause time, use memory efficiently, and maintain good data locality.

➢ Approaches for Identifying Garbage / Methods of Garbage Detection

Garbage Collector (GC) needs to determine which objects are still being used and which objects are no longer needed. An object that is no longer needed is called garbage and its memory can be reclaimed. There are two common methods:

1 Reachability 2. Reference counting

1 Reachability

In GC, reachability determines whether a piece of memory (an object) can still be used by the program. An object is considered:

- Reachable → If your program can access it in any way via references from active variables, the call stack, or other reachable objects.
- Unreachable → If there is no way for the program to access it, then it can be safely garbage collected

2. Reference Counting

Reference counting keeps track of the number of references pointing to each object.

- If an object has one or more references → it is considered alive.
- If its reference count becomes zero → it is considered garbage.

Operation	**Description**	**Effect on Reference Count**
Object creation	When a new object is allocated	count = 1
Assignment	When another variable points to the object	count++
Reference removal / variable reassignment	When a variable stops referring to it	count--
Deallocation	When count == 0	Object is free
Parameter passing	When an object passed	Count++

Reference counting has two main disadvantages: it cannot collect unreachable, cyclic data structures, and it is expensive.

- **Mutator**

The running program itself is called the *mutator*. Mutator is a part of program that allocated and manipulate objects in memory

The mutator is the running program that:

- Creates objects A and B.
- Stores them in heap memory.
- Creates references between objects.
- Changes references during execution.
- Makes some objects reachable or unreachable.

The mutator performs the following operations on objects:

Mutator Operation	*Example*	*Effect on Reachability*	*Impact on GC*
1. Object Allocation	A = new Node();	Adds a new reachable object (because it's referenced by A)	GC must track this object as live until A loses reference
2. Reference Assignment	B.next = A;	Creates a new link → increases reachability (A now reachable from B)	GC must consider new path during mark phase
3. Reference Overwrite	A = new Node(); (old object lost)	Removes previous link, possibly making old object unreachable	Old object may become garbage
4. Null Assignment	A = null;	Removes last reference → object becomes unreachable	GC can safely reclaim memory

➢ Trace-Based Collection

Garbage collection involves the collection of garbage as it is being created; trace-based garbage collection involves periodically collecting garbage.

Trace-based garbage collectors will run at regular intervals to find unreachable objects (garbage) and regain their space

Trace-based garbage collection algorithms determine which objects are *alive* (in use) and which are *garbage* (unreachable) by tracing all references starting from a set of root objects.

Starts from GC Roots

- The GC begins with special references called roots.
- Examples: active local variables, global/static variables, and CPU registers.

Traces Object References

- The GC follows references from the roots to other objects.
- It continues tracing until all reachable objects are identified.

Marks Live Objects

- Objects that can be reached from the roots are marked as live.

Unreachable Objects Are Garbage

- Objects that cannot be reached from any root are considered garbage.

Garbage Is Reclaimed

- The memory occupied by unreachable objects is recovered and made available for future allocations.

Can Detect Cyclic Garbage

- Unlike simple reference counting, tracing can identify objects that reference each other but are no longer reachable from the roots.

Trace-based GC Algorithms

Trace-based GC algorithms identify garbage by starting from GC roots and tracing all reachable objects. The main algorithms are:

1. Mark-and-Sweep Algorithm

The Mark-and-Sweep algorithm works in two main phases: Mark and Sweep.

Mark Phase

Start from the GC roots.
Trace all objects reachable from the roots.
Mark every reachable object as live.

Sweep Phase

Scan the entire heap.
Reclaim all unmarked objects, since they are unreachable.
The reclaimed memory is returned to the free-memory pool.

Advantage: Can detect and reclaim cyclic garbage.

Disadvantage: Can cause heap fragmentation because live objects are not moved.

Process

GC Roots → Mark reachable objects → Sweep unmarked objects → Reclaim memory

2. Mark-and-Compact

The Mark-and-Compact algorithm first identifies live objects and then moves them together to eliminate fragmentation.

Mark Phase

Start from the GC roots.

Trace all reachable objects.

Mark every reachable object as live.

Compact Phase

Move the live objects together into one continuous memory area.

Update references to the objects after they are moved.

The remaining memory becomes one large free block.

Advantage Reduces or eliminates external fragmentation.

Provides a large contiguous free-memory area.

Disadvantage : Moving objects and updating references requires additional processing time.

Process

GC Roots → Mark live objects → Move/compact live objects → Update references → Reclaim free space

3. Copying Collection

It is a trace-based garbage collector that uses two memory regions, commonly called from-space and to-space.

Working

Divide the heap into two regions.

Allocate objects in the active from-space.

When garbage collection starts, trace the reachable objects.

Copy live objects from from-space to to-space.

Update references to their new locations.

After copying, the entire old from-space can be reclaimed.

The roles of the two regions are then exchanged.

Advantage : Eliminates external fragmentation.

Allocation is fast because free space is contiguous.

Disadvantage : Requires additional memory because two regions are maintained.

Live objects must be copied during collection.

Process

From-space → Trace reachable objects → Copy live objects to to-space → Update references → Reclaim from-space

4. Baker's incremental copying

Baker's algorithm is an incremental approach to garbage collection. It divides garbage collection work into smaller steps so that the program does not need to be stopped for a long period.

Working

The collector traces reachable objects incrementally.

Objects are marked as they are discovered.

Garbage objects are identified and reclaimed gradually.

The collector performs part of the GC work while the program continues to execute.

In a normal copying collector:	In **incremental copying**:
Stop program ↓ Copy ALL live objects ↓ Resume program	Copy some objects ↓ Let program continue ↓ Copy some more objects ↓ Let program continue ↓ Repeat until copying is complete

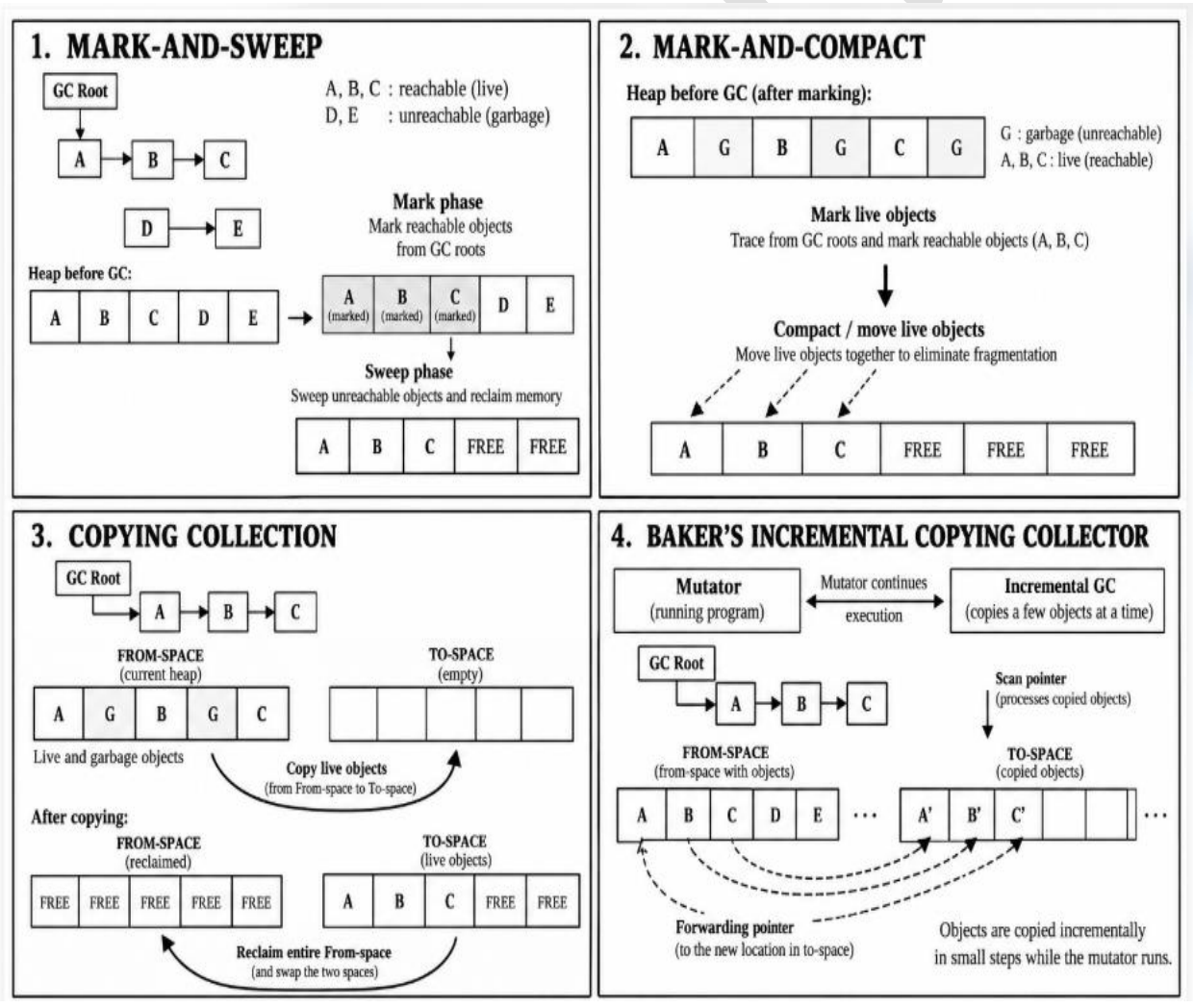
Advantage Reduces long pause times of the application.
Suitable for systems where continuous program execution is important.

Disadvantage More complex to implement.
The collector may require additional processing while the program is running.

Process

GC Roots → Incrementally trace reachable objects → Mark live objects → Reclaim garbage

The main difference between these algorithms is how the identified live and garbage objects are handled after tracing.



Type Safe

- A language is considered type-safe if the type of any data component can be determined, either at compile time (statically typed, e.g., ML) or at runtime (dynamically typed, e.g., Java). This allows the garbage collector to know which memory locations might contain pointers.
- If a language is neither statically nor dynamically type safe, then it is said to be unsafe. Unsafe languages, which unfortunately include some of the most important languages such as C and C++, are bad candidates for automatic garbage collection.

CODE GENERATOR

Issues in the Design of a Code Generator

The code generator transforms the intermediate representation of source code into a machine-readable format. The correct code should be generated using a code generator.

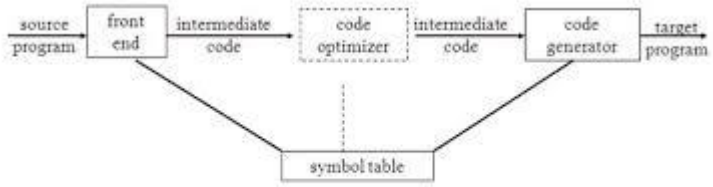


Fig. 4.1 Position of code generator

The main issues in the design of a code generator are:

- Input to the code generator
- Target program
- Instruction selection
- Register allocation
- Evaluation order

1. Input to the code generator

Intermediate codes may be represented mainly in quadruples, triples, indirect triples, postfix notation, syntax trees, DAGs(Directed Acyclic Graph), etc. The code generation step assumes that the input is free of all syntactic and semantic mistakes, that all essential type checking has been performed, and that type-conversion operators have been introduced where needed.

2. Target program

The code generator's output is the target program. The result could be:

- **Assembly language:** It allows subprograms to be separately compiled.
- **Relocatable machine language:** It simplifies the code generating process. Suitable if no. of instructions are less. Programs can compile and execute quickly.
- **Absolute machine language:** It can be stored in a specific position in memory and run immediately. It is suitable if program linked with other modules

The target machine can have either CISC or RISC processor architecture.

3. Instruction selection

In the Instruction selection, the design issues in the code generator program's efficiency will be improved by selecting the optimum instructions.

Instruction selection is optional if we don't care about the target program's efficiency.

The relevant three-address statements, for example, would be translated into the following code sequence:

P:=Q+R

S:=P+T

```
MOV Q, R0
ADD R, R0
MOV R0, P
MOV P, R0
ADD T, R0
MOV R0, S
```

The fourth sentence is unnecessary since the P value is loaded again in that statement already stored. It results in an inefficient code sequence. A given intermediate representation can be translated into several distinct code sequences, each with considerable cost differences. Previous knowledge of instruction cost is required to build good sequences, yet reliable cost information is difficult to forecast.

4. Register allocation

In the Register allocation, design issues in the code generator can be accessed faster than memory. The instructions involving operands in the register are shorter and faster than those involved in memory operands.

The following sub-problems arise when we use registers:

- **Register allocation:** In register allocation, we select the set of variables that will reside in the register.
- **Register assignment:** In the Register assignment, we pick the register that contains a variable.

Certain machines require even-odd pairs of registers for some operands and results.

5. Evaluation order

The code generator determines the order in which the instructions are executed. The target code's efficiency is influenced by order of computations. Many computational orders will only require a few registers to store interim results. However, choosing the best order is a completely challenging task in the general case.

➢ Basic Blocks and Flow Graphs

Basic blocks and flow graphs are used in compiler design to represent and analyze program code for *optimization.*

A compiler first converts source code into an intermediate code, then partitions this intermediate code into a series of basic blocks.

Basic blocks are sequences of consecutive instructions with a single entry and exit point, while a control flow graph represents the flow of execution between these blocks using a directed graph.

Basic blocks

In compiler design, a basic block is a straight-line piece of code that has only one entry point and one exit point.

Characteristics of Basic Blocks:

- Single Entry Point: Control can enter a basic block only through its first instruction.
- Single Exit Point: Control leaves the block only from its last instruction, either by executing the next instruction or through a jump.
- Sequential Execution: Instructions within a basic block execute in sequence, unless the last instruction transfers control elsewhere.
- No Internal Branching: There are no jump or branch instructions in the middle of a basic block.
- No Jump into the Middle: No jump instruction from another block can target an instruction inside the block, except its first instruction.
- Consecutive Instructions: All instructions in a basic block appear consecutively in the intermediate code.
- Used for Optimization: Basic blocks help compilers perform local optimizations, such as constant folding, common subexpression elimination, and dead code elimination.

Algorithm of Basic Block in Compiler Design / Partition into basic blocks

Input**:** Sequence of three address statements.

Output**:** A sequence of basic blocks.

The algorithm for identifying basic blocks in three-address code consists of two main steps:

- Step 1: Identify Leaders.

A leader is the first instruction of a basic block. The following rules are used to identify leaders:

1. The first instruction of the intermediate code is a leader.
2. Any instruction that is the target of a conditional or unconditional jump is a leader.
3. Any instruction that immediately follows a conditional or unconditional jump is a leader.

- Step 2: Form Basic Blocks

For each leader:

1. The leader becomes the first instruction of a basic block.
2. The block contains all consecutive instructions up to, but not including, the next leader.
3. The last block contains all remaining instructions.

Flow graph / Control Flow Graph / CFG

A flow graph is a directed graph that represents the flow of control among basic blocks in a program. It helps the compiler understand the possible execution paths and perform code optimization.

Components of a Flow Graph

- Nodes: Each node represents a basic block.
- Directed edges: An edge from Bi to Bj indicates that control can pass from basic block Bi to basic block Bj.
- Entry node: Represents the first basic block of the program.

Example

Three-address code:

1. a = b + c
2. d = a * 2
3. if d > 10 goto L1
4. e = d + 5
5. L1: f = e – 1

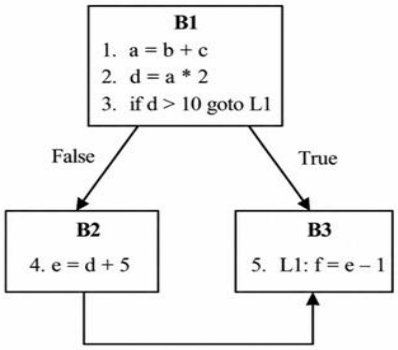
Basic blocks:

B1:
1. a = b + c
2. d = a * 2
3. if d > 10 goto L1

B2:
4. e = d + 5

B3:
5. L1: f = e – 1

Control Flow Graph:



- B1: Instructions 1, 2, 3
- B2: Instruction 4
- B3: Instruction 5

Instruction 1 is the first instruction, so it is a leader.

Instruction 3 is a conditional jump, but it can remain inside B1 because it is the last instruction of the block.

Instruction 4 immediately follows the conditional jump, so it is a leader.

Instruction 5 is the target of the jump (L1), so it is also a leader.

Example

```
s = 0;
i = 0;
n = 10;
while (i < n) {
  if (a != b)
      s = s + (i * 4);
  else
      s = s + i;
  i = i + 1;
}
```

Three-address code

1	s := 0
2	i := 0
3	n := 10
4	L1: t1 := n – i
5	ifz t1 goto L2
6	t2 := a – b
7	ifz t2 goto L3
8	t3 := i * 4
9	s := s + t3
10	goto L4
11	L3: s := s + i
12	L4: i := i + 1
13	goto L1
14	L2: end

Basic Blocks

B1:
1. s := 0
2. i := 0
3. n := 10

B2:
4. L1: t1 := n – i
5. ifz t1 goto L2

B3:
6. t2 := a – b
7. ifz t2 goto L3

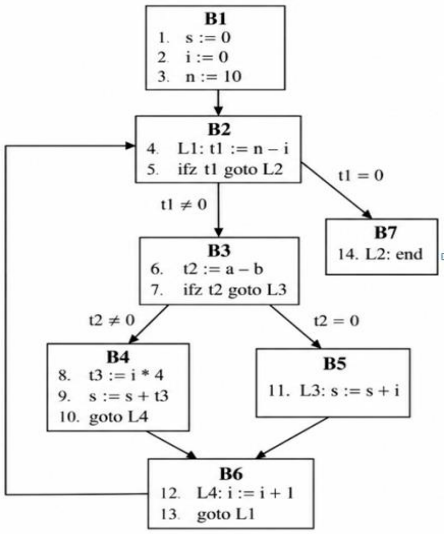
B4:
8. t3 := i * 4
9. s := s + t3
10. goto L4

B5:
11. L3: s := s + i

B6:
12. L4: i := i + 1
13. goto L1

B7:
14. L2: end

Control Flow Graph

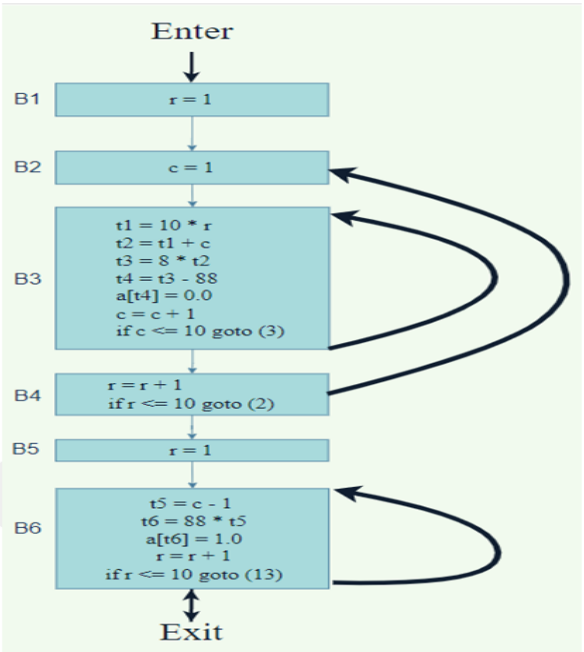


Example

```
Source code
for r from 1 to 10 do
     for c from 1 to 10 do
          a [ r, c ] |= 0.0;
for r from 1 to 10 do
       a [ r, c ] = 1.0;
```

TAC

1) r = 1
2) c = 1
3) t1 = 10 * r
4) t2 = t1 + c
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) c = c + 1
9) if c <= 10 goto (3)
10) r = r + 1
11) if r <= 10 goto (2)
12) r = 1
13) t5 = c - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) r = r + 1
17) if r <= 10 goto (13)



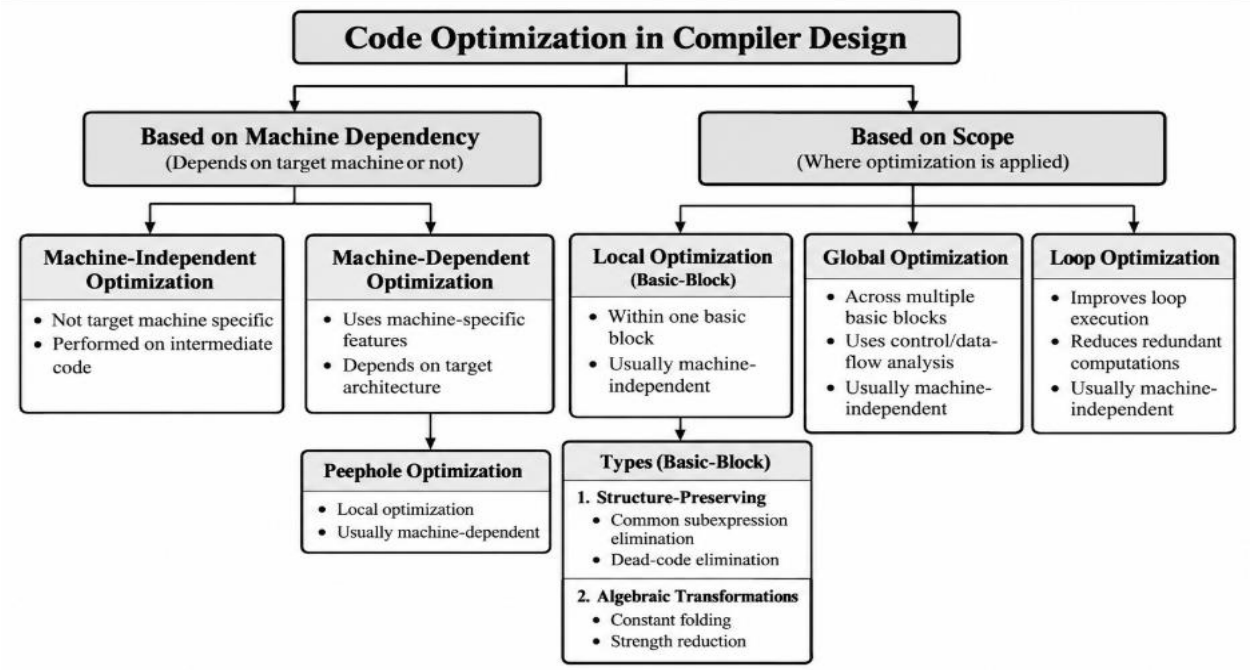
➢ Optimization

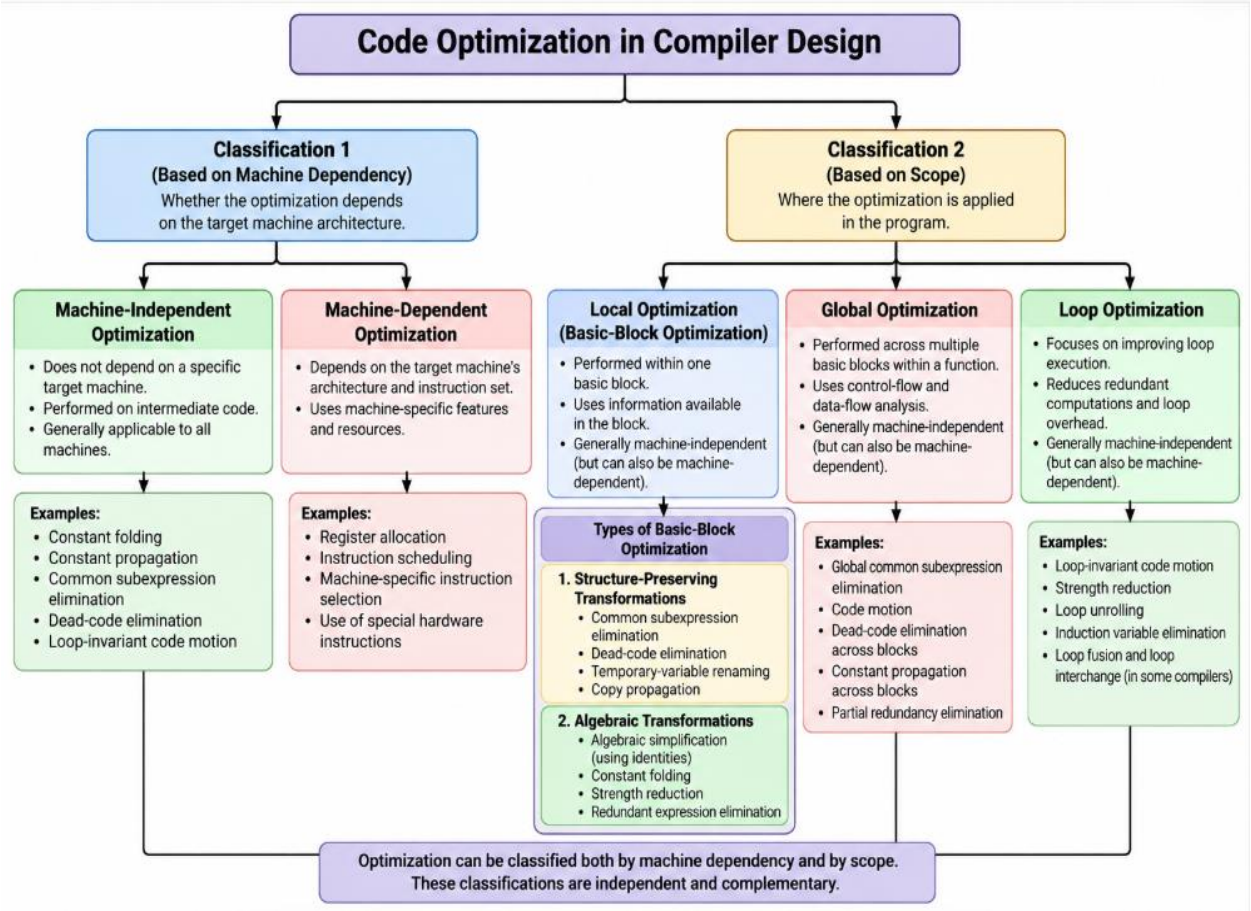
Code optimization is the process of transforming a program into an equivalent program that executes more efficiently, without changing its output or functionality.

The main goal is to improve program performance by reducing execution time, memory usage, or the number of instructions. It involves eliminating redundancies and unnecessary operations .

Optimization of basic blocks can be machine-dependent or machine-independent.

Types of Code Optimization





Steps before optimization:

1) Source program should be converted to Intermediate code
2) Basic blocks construction 3) Generating flow graph 4) Apply optimization

The optimization process can be broadly divided into two types

1. Based on machine dependency

a) **Machine Independent Optimization-** This code optimization process optimizes the intermediate code to produce a better target code. it is not depending on a specific processor or machine architecture.
   - Improve the intermediate code to get a better target code as output
   - Intermediate code transformed doesn't involve any CPU registers or absolute memory locations
   Ex : structured preserving transformation and Algebraic transformation

b) **Machine Dependent Optimization-** This optimization is performed after the target code has been created and converted according to the target machine architecture.
   Ex: PeePhole Opimization, Register Allocation
   - Improve the target code according to the target machine architecture
   - Involves CPU registers and may have absolute memory references.
   - Put efforts to take maximum advantage of the memory

2. Based on Scope

   a) Local optimization
   b) Global optimization
   c) Loop optimization

a) Local Optimization

Local optimization is a code optimization technique performed within a single basic block.
No need to analyze whole procedure body
It is simple
Ex : Common subexpression elimination, Constant folding, Dead-code elimination

b) Global Optimization

Global optimization is a code optimization technique performed across multiple basic blocks within a function or procedure. It considers the flow of control between basic blocks to improve code efficiency.
Example: Reusing a previously calculated expression across blocks when its operands have not changed.

c) Loop Optimization

Loop optimization is a code optimization technique used to improve the execution speed and efficiency of loops by reducing redundant computations, minimizing loop overhead, and improving resource utilization.
Loop optimization includes the following strategies:

   a) Code motion & Frequency Reduction
   b) Induction variable elimination
   c) Loop merging/combining
   d) Loop Unrolling

➢ Optimization of Basic Blocks / Transformation of Basic Blocks

There are two types of basic block optimizations.

They are :

1. Structure-Preserving Transformations
2. Algebraic Transformations

1. Structure-Preserving Transformations:

Structure-preserving transformations are techniques used in basic-block optimization to improve intermediate code by eliminating redundant computations and unnecessary instructions while preserving the program's meaning.

The structure-preserving transformation on basic blocks includes:

a) Dead Code Elimination
b) Common Subexpression Elimination
c) Renaming of Temporary variables
d) Interchange of two independent adjacent statements

a) Common sub-expression elimination:

Common sub expressions need not be computed over and over again.

Instead they can be computed once and kept in store from where it's referenced.

Example:

suppose the basic block is a: =b+c b: =a-d c: =b+c d: =a-d

Basic block can be transformed to a: = b+c b: = a - d **c: = a d: =b**

b) Dead code elimination:

Dead code is defined as that part of the code that never executes during the program execution or whose results never used. So, for optimization, such code or dead code is eliminated.

Eliminating the dead code increases the speed of the program as the compiler does not have to translate the dead code.

Example 1	Example 2

a = 10 // dead code, not used
a = 20
print(a)
	int main()
	{ x = 2
	if (x > 2)
	cout << "code"; // Dead code
	else
	cout << "Optimization";
	return 0;
	}

c) Renaming of temporary variables:

Renaming of temporary variables does not directly reduce execution time. Its main benefit is making data dependencies clearer and enabling other optimizations.

Before renaming	After renaming	After dead-code elimination:
1. t1 = a + b 2. t1 = c * d 3. e = t1 + 5	1. t1 = a + b 2. t2 = c * d 3. e = t2 + 5	1. t2 = c * d 2. e = t2 + 5

d) Interchange of two independent adjacent statements:

It is a structure-preserving transformation in which two consecutive statements in a basic block are swapped when they are *independent* of each other, without changing the program's result.

Interchanging independent adjacent statements does not directly reduce the number of instructions. It allows better instruction scheduling and may improve processor utilization and execution speed.

Before interchange:	After interchange:
1. a = b + c	1. x = y * z
2. x = y * z	2. a = b + c
3. p = a + 5	3. p = a + 5

2. Algebraic Transformations:

Algebraic transformations are optimization techniques that use algebraic rules and mathematical identities to simplify expressions and make the code more efficient. They are commonly used in basic-block (local) optimization.

Some of the algebraic transformation on basic blocks includes:

a) Constant Folding
b) Copy Propagation
c) Strength Reduction
d) Algebraic simplicfiation

a) Constant Folding:

Combines and evaluates multiple constants into a single value whose value can be computed before executing the code.

Example : x = 2 * 3 + y ⇒ x = 6 + y (Optimized code)

Example : a = 22 / 7*r *r ➔ a = 3.14 * r * r

b) Copy Propagation

Copy propagation can be used in both local and global optimization.

It is of two types, Variable Propagation, and Constant Propagation.

Variable Propagation: replaces a variable with another variable whose value it copies.

Example y = x z = y + 2 optimized code z = x + 2

Constant Propagation:

x = 3 z = x + a Optimized code z = 3 + a

Example : pi =3.14

Area = pi * r* r ➔ area = 3.14 * r * r

c) Strength Reduction

Replace expensive statement/ instruction with cheaper ones.

x = 2 * y (costly) ⇒ x = y + y (cheaper)

x = y** 2 (costly) ⇒ x = y *y (cheaper)

d) Algebraic Simplification

Uses mathematical identities to remove unnecessary operations.

Before optimization After optimization

```
x = y + 0        x = y
x = y * 1        x = y
x = y - 0        x = y
x = y * 0        x = 0
x = y / 1        x = y
```

Feature	Constant Folding	Constant Propagation
Core Action	Combines and evaluates multiple constants into a single value.	Substitutes a known constant value into variables later in the code.
Focus	Expressions (e.g., `3 + 5`)	Variables (e.g., replacing `x` with `5`)
Example	`int x = 2 + 3;` becomes `int x = 5;`	If `x = 5`, then `y = x + 1` becomes `y = 5 + 1`

- ***"Copy propagation Leads to Dead code"***

Copy propagation can make a statement unnecessary, allowing the compiler to remove it through dead-code elimination.

When copy propagation replaces a variable by its source, the original copy assignment may become useless because the copy variable is no longer used.

Example

Before Copy Propagation:	After Copy Propagation:

1. x = y;
2. z = x + 5;
	1. x = y; // (copy statement)
	2. z = y + 5; // x replaced with y
	Now, x is no longer used anywhere in the program.
	Therefore, statement 1 (x = y) is dead code.

- Loop Optimization

Loop optimization is a code optimization technique that improves the efficiency of loops by reducing unnecessary computations, minimizing loop overhead, and improving execution speed.

Loop optimization includes the following strategies:

- Code motion & Frequency Reduction
- Induction variable elimination
- Loop merging/combining
- Loop Unrolling

a. Code motion

It is a technique which moves the code outside the loop if it won't have any difference if it is executed out side or inside loop

int main()
{ for (i = 0; i < n; i++) {
x = 10;
y = y + i;
}
	After optimization
	int main()
	{ x = 10;
	for (i = 0; i < n; i++)
	y = y + i;
	return 0; }

b. Induction Variable Elimination:

Eliminate various unnecessary induction variables used in the loop. A variable is induction variable if it changes its value for every iteration in loop

// Program with multiple induction variables
 	
i1 = 0;
i2 = 0;
for (i = 0; i < n; i++) {
A[i1++] = B[i2++];
}
	// Program with one induction variable
	
	for (i = 0; i < n; i++) {
	A[i] = B[i]; // Only one induction variable
	}

c. Loop Merging/Combining:

Loop merging (also called loop fusion) is a loop optimization technique in which two or more loops with the same iteration range are combined into a single loop to reduce loop overhead.

// Program with multiple loops	// one loop when multiple loops are merged

for (i = 0; i < n; i++)
A[i] = i + 1;
for (j = 0; j < n; j++)
B[j] = j - 1;
	for (i = 0; i < n; i++) {
	A[i] = i + 1;
	B[i] = i - 1;
	}

d. Loop Unrolling:

If there exists simple code which can reduce the number of times the loop executes then, the loop can be replaced with these codes.

// Program with loops	// Program with simple code without loops
for (i = 0; i < 3; i++) cout << "Cd";	cout << "Cd"; cout << "Cd"; cout << "Cd";

➢ Simple Code Generator

A simple code generator is the final phase of a compiler that converts the intermediate representation (usually Three-Address Code, TAC) into target assembly or machine code.

Its main objective is to generate correct and efficient target code while minimizing unnecessary instructions.

Next-use information :

1. The code generator uses next-use information to decide whether to keep a value in a register or store it in memory so that the register can be reused.
2. It uses the getreg() function to select a suitable register for an operand or result.
3. Before generating target code, it checks whether the operands are already available in registers. If not, it loads them into registers.
4. After performing an operation, the result is kept in a register if it is needed for further computation.
5. When a value is no longer needed, its register can be reused for another computation.
6. At a procedure call or the end of a basic block, the code generator saves any values that must be preserved.

Data structures for the Simple code generator algorithm

Data structure	Purpose
Register descriptor	Records which variables currently reside in each register.
Address descriptor	Records where the current value of each variable is available, such as registers or memory.
Next-use information	Helps decide whether a variable will be needed again and whether its register can be reused.

A Code-Generation Algorithm:

Input : Sequence of 3-address statements from a basic block.

Output: Assembly language code

For each three-address statement of the form `x := y op z`, perform the following steps:

1. Call `getreg()` to select a suitable location `L` to store the result of `y op z`.

2. Check the address descriptor of `y` to find its current location `y'`. If `y` is available in both a register and memory, prefer the register. If `y` is not already in `L`, generate a `MOV` instruction **MOV y',L** to copy `y` into `L` .

3. Generate the target instruction **OP z',L** where `z'` is the current location of `z`. If `z` is available in both a register and memory, prefer the register. Update the address descriptor of `x` to indicate that its value is in `L`, and remove outdated locations of `x` from the descriptors.

4. If `y` or `z` has no further use and is not live at the end of the basic block, update the register descriptor to indicate that its register can be reused.

Algorithm getreg ( )

Input: Request for a register

Output: A register or the memory location

1. If *y* is stored in a register *R* and *R* only holds the value *y*, and *y* has no next use, then return *R*; Update address descriptor: value *y* no longer in *R*
2. Else, return a new empty register if available
3. Else, find an occupied register *R*; Store contents (register spill) by generating MOV *R*,*M* for every *M* in address descriptor of *y*; Return register *R*
4. Else Return a memory location

Example Generating Code for Assignment Statements:

• The assignment d : = (a-b) + (a-c) + (a-c) might be translated into the following three-address code sequence:

Statement	Code Generated	Register Descriptors	Address Descriptors
Initial state	—	Registers empty	a: M b: M c: M t: M u: M v: M d: M
1. t=a-b	`MOV a, R0`	R0: a	**a: M, R0** b: M c: M t: M u: M v: M d: M
	`SUB b, R0`	R0: t	a: M b: M c: M **t: R0** u: M v: M d: M
2. u=a-c	`MOV a, R1`	R0: t; R1: a	**a: M, R1** b: M c: M **t: R0** u: M v: M d: M
	`SUB c, R1`	R0: t; R1: u	**a: M** b: M c: M **t: R0** **u: R1** v: M d: M
3. v=t+u	`ADD R1, R0`	R0: v; R1: u	a: M b: M c: M **t: M** **u: R1** **v: R0** d: M
4. d=v+u	`ADD R1, R0`	R0: d; R1: u	a: M b: M c: M t: M **u: R1** v: M **d: R0**
	`MOV R0, d`	R0: d; R1: u	a: M b: M c: M t: M u: R1 v: M **d: M, R0**

➢ Peephole optimization

- Peephole optimization falls under machine-dependent optimization. Machine-dependent optimization takes place *after the target code is generated* and adapted to the machine's architecture. It uses CPU registers and may use absolute memory references instead of relative ones.
- Peephole optimization is a *subset* of local optimization. It's a specific method used to achieve local optimization.
- Local optimization works on a broader scope, like a basic block, while peephole optimization is limited to a very small, fixed "window" of instructions. Peephole is a small moving window on the target systems
- Peephole optimization is simpler because it only needs to analyze a small, localized set of instructions at a time.
- It is applied repeatedly to small sections of code.

Types of Peephole Optimization

1. Redundant instruction elimination
2. Constant folding
3. Algebraic simplification
4. Strength reduction
5. Flow-of-control optimization
6. Unreachable code elimination
7. Machine idiom replacement

- **Flow-of-control optimization :** Simplifies unnecessary jumps.

Example :

Before	After optimized
JMP L1 L1: JMP L2	JMP L2

- **Unreachable code elimination :** Removes instructions that can never execute.

Before:	After:
JMP L1 MOV R0, a L1: ADD R0, b	JMP L1 L1: ADD R0, b

- **Machine Idiom Replacement**

Replaces a sequence of instructions with a shorter or faster instruction supported by the target machine.

Example:

Before:	After, if supported by the target machine:
MUL R0, 2	SHL R0, 1 Here, multiplication by 2 is replaced with a left shift by one bit.
Ex2: Add #1, R	Inc R

➢ Register Allocation and Assignment

Registers are *faster* but *limited in number*

1. Which values should be kept in registers instead of memory → Register Allocation
2. Which specific register should hold each value → Register Assignment

Register allocation

- It is one of the final and most crucial optimization phases in compiler design. Because access to CPU registers is exponentially faster than access to main memory, keeping frequently used variables in registers drastically improves instruction execution speed.
- **Register allocation** is the process of deciding *which variables or temporary values* will be stored in registers at any given point during program execution. Since a processor has a limited number of registers, the compiler must select which variables deserve them.
- Register allocation depends on:
  – Size of live range
  – Number of uses/definitions
  – Frequency of execution
  – Number of loads/stores needed.

  Example: Three-address code (TAC):

  t1 = a + b

  t2 = t1 * c

  t3 = t2 - d

  The compiler may decide to store t1 and t2 in registers rather than memory to avoid repeated memory accesses.
- Purpose: - Minimize memory accesses.
  - Reduce execution time.
  - Make efficient use of limited registers.
- The major register allocation methods are: local allocation, global allocation, graph coloring, and linear scan , Priority-based allocation

Register Assignment

- Register assignment is the process of assigning specific physical CPU registers to the variables or temporary values selected during register allocation.
- For example, after deciding that t1 and t2 should be stored in registers, the compiler assigns actual registers to them.

Variable	Assigned register	
t1	R1	
t2	R2	
t3	R1	Here, t1 and t3 can share R1 if their live ranges do not overlap.

Register Spilling

- Register spilling is the process in compilers where a variable's value is moved from a CPU register to main memory, or RAM, because there are not enough registers to hold all the variables in use.
- This happens when the number of variables exceeds the number of available registers, forcing the compiler to choose some variables to "spill" to memory.
- Spilling requires the compiler to generate extra instructions to load and store values, which increases code size and execution time.

➢ Register Allocation Methods

Based on Scope of allocation , it can be 2 types

a) Local register allocation

- The compiler allocates registers for temporary variables used inside one basic block without considering other blocks. It follows top-down approach.
- Assign registers to the most heavily used variables
    - Traverse the block
    - Count uses
    - Use count as a priority function
- Assign registers to higher priority variables first

b) Global register allocation:

- Local allocation does not take into account that some instructions (e.g. those in loops) execute more frequently. A variable used in several basic blocks may be kept in a register across block boundaries.
- To find out the live range(s) of each variable and the area(s) where the variable is used/defined global allocation is needed. Cost of spilling will depend on frequencies and locations of uses.
- Methods used for global register allocation are Graph coloring , usage counts, Live range analysis

Classification based on the allocation algorithm

a) Graph Coloring

- Construct an interference graph using the live-variable information across basic blocks.
- An interference graph tells us which variables cannot share registers, and graph coloring tells us how to assign registers to them.
- Nodes: Represent variables whose values need to be stored.
- Edges: Connect variables that are live at the same time and therefore cannot share a register.
- Colors: Represent available CPU registers.
- Rule: Two connected nodes must have different colors (registers).
- assign registers (colors) to nodes such that connected nodes receive different registers, if registers are insufficient, spill selected variables to memory.

b) Linear Scan

- Determine the live interval of each variable across basic blocks.
- Sort the intervals by their starting points.
- Assign available registers to intervals and release registers when intervals expire.
- Spill variables when no suitable register is available.

Example for Graph coloring

Idea: Variables whose live ranges overlap cannot be assigned the same register.

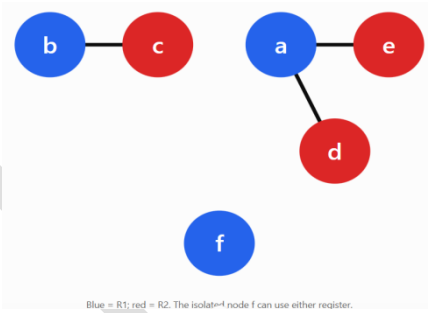
Consider the three-address code:

1. a = b + c
2. d = a + e
3. f = d + a

Assume the compiler has 2 available registers: R1 and R2.

Identify live variables

- a is created in statement 1 and used in statements 2 and 3.
- d is created in statement 2 and used in statement 3.
- f is created in statement 3 and is the final result.



The live ranges of a and d overlap, so they cannot share a register.

Draw one node for each variable. Connect only the pairs that interfere.

Assume the compiler has 2 available registers: R1 and R2.

Blue = R1 Red = R2

In statement 1, b and c are needed simultaneously, so they interfere.b, c: connected, so they need different registers.

In statement 2, a and e are needed simultaneously, so they interfere. a,e: connected, so they need different registers.

In statement 3, d and a are needed simultaneously, so they interfere. a,d: connected, so they need different registers.

f: isolated, No edge connects f to either a or d, because f is defined at the final statement and can reuse a register after both operands have been read.

Result: Graph coloring assigns registers by ensuring that interfering variables receive different registers.

Example for Global Register Allocation by Linear Scan

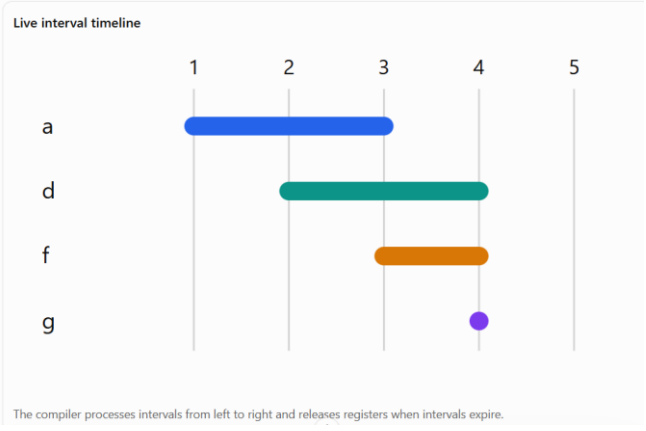
Consider the following three-address code:

1. a = b + c
2. d = a + e
3. f = d + a
4. g = f + d

Assume two registers, R0 and R1, are available.

consider the live intervals of the variables a, d, f, g

Variable	Start	End	Live interval
a	1	3	[1, 3]
d	2	4	[2, 4]
f	3	4	[3, 4]
g	4	4	[4, 4]



At instruction 1, **a** receives R0.
At instruction 2, **d** needs a register while a is still live, so **d** receives R1.
At instruction 3, **a** reaches its last use. Its register can be released after that use. The compiler can then reuse R0 for **f**, depending on the precise instruction-level live ranges.
At instruction 4, registers are reassigned as variables become dead and new values are produced.
The important rule is that two variables with overlapping live intervals cannot use the same register.

Usage Counts

Usage count is a heuristic method used in global register allocation to prioritize which variables to assign to registers by estimating the cost-benefit of an allocation
Registers are limited. So If we have more variables than registers, we must choose which ones to keep in registers and which to spill to memory.
Variables with higher usage counts are *kept in registers longer*.
Variables with low usage counts can be *spilled to memory*.

Example : Consider this code:
1. a = b + c
2. d = a + e
3. f = d + a
4. g = f + d
5. h = a + g

Count how many times each variable appears as an operand (right-hand side).

Variable	Instructions where used	Usage count
a	2, 3, 5	3
d	3, 4	2
f	4	1
g	5	1

Usage counts important to know
- Frequently used variables: Keeping them in registers can reduce repeated memory accesses.
- Less frequently used variables: They may be better candidates for spilling when registers are insufficient.
- Register allocation decisions: Usage counts can help prioritize variables, particularly when choosing which live interval to spill.

It is calculated for each variable by summing its usage frequency in a basic block and doubling its "liveness" value (whether the variable is needed after the block).
The formula is often expressed as **usage_count = use(x, b) + 2 * live(x, b)**, where use(x, b) is the number of times variable x is used in block b prior to any definition, and live(x, b) is 1 if x is live on exit from b, and 0 otherwise.